

TWO-DIMENSIONAL ALGORITHMS

3.1 Line Drawing Algorithms

3.1.1 DDA

3.1.2 Bresenham's Algorithm

3.2 Circle Generation Algorithm

3.3 Ellipse Generation Algorithms

3.4 Area Filling-Scan Line Algorithm

3.5 Boundary Fill Techniques

3.6 Flood Fill Techniques

Scan Conversion | Rasterization

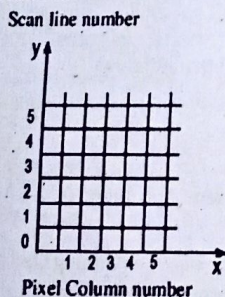
The process of representing a continuous graphical objects as a collection of discrete pixels by identifying their locations and setting them On is called scan conversion

- The process of determining which pixels will provide the best approximation to the desired line is properly known as rasterization.
- Combined with the process of rendering the picture in scan line order it is known as scan conversion.
- **Rasterization:** Process of determining which pixels give the best approximation to a desired line on the screen.
- Scan Conversion-Digitizing a picture definition given in an application program into a set of pixel intensity values for storage in the frame buffer.
- The basic building block of pictures is referred to as output primitives.
- Output primitives are the geometric structures that can be used to describe the shape and color of the objects.
- They include character, strings and geometric entities such as points, straight lines, circle, polygons etc. (Point and Lin are the most basic components of Picture)
- In a raster display system, a picture is specified by set of intensities for the pixel positions in the display.
- We generate images by turning the pixels ON or OFF, so it is necessary to represent an object (continuous) as a collection of discrete pixels.
- Hence the process of determining the appropriate pixels for representing a picture (or graphics object) is called rasterization.

- The process of conversion of the rasterized picture i.e. picture definition, stored in the frame buffer into a set of pixel intensity values for the rigid display pattern is called scan conversion. This is done by display processor.

Point Scan Conversion

- A mathematical point (x, y) where x and y are real numbers within an image area, needs to be scan converted to a pixel at location (x', y') .
- So point plotting is accomplished by converting a single coordinate position into appropriate operation for the output device in use. Example: For CRT, Electron beam can be turned on for illuminate the screen phosphor or off at the selected location on the display technology.
- Now the question is how the electron beam is positioned depends upon the display technology?
- In case of raster system, a point can be plotted by setting the bit value corresponding to a specified screen position within the frame buffer to one. This is done by simply turning on the electron beam at that point.
- But in case of random system, it stores the point plotting instruction in the display list file. The coordinate values in this file are converted to deflection voltage that positions the electron beam at the screen location to be plotted.
- In raster system, line drawing is accomplished by calculating intermediate positions along the line path between two specified points. Then the output device is directed to fill in these positions between endpoints.
- But in case of random system (example: vector pen plotter), line drawing is accomplished by retrieving line drawing commands from the display list.
- Scan lines are numbered consecutively from 0, starting the bottom of the screen; the pixel columns are numbered from 0, from left to right across each scan line.



Line Scan Conversion

- The Cartesian slope-intercept equation for a straight line is $y = mx + c$ (i)

where,

m = slope of line and c = y intercept

- When Use two endpoints of a line are given as (x_1, y_1) and (x_2, y_2) as shown in figure below, we can determine the value of slope m and intercept c with the following calculation

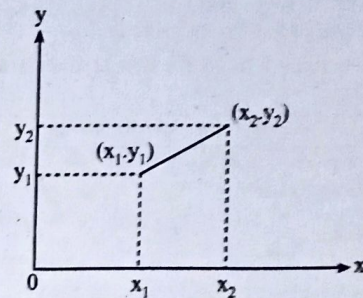


Fig: Line path between end point position (x_1, y_1) and (x_2, y_2)

$$m = \frac{(y_2 - y_1)}{(x_2 - x_1)} \quad \text{..... (ii)}$$

Now, for any point (x_k, y_k) , i becomes,

$$y_k = m \cdot x_k + c \quad \text{..... (iii)}$$

Again for any point (x_{k+1}, y_{k+1}) , i become

$$y_{k+1} = m \cdot x_{k+1} + c \quad \text{..... (iv)}$$

Subtraction iii from iv we get,

$$y_{k+1} - y_k = m \cdot x_{k+1} + c - m \cdot x_k - c$$

$$y_{k+1} - y_k = m \cdot (x_{k+1} - x_k)$$

i.e. $\Delta y = m \Delta x$, where, Δy and Δx are small increment in x and y respectively.

$$\text{Therefore, } m = \Delta y / \Delta x \quad \text{..... (v)}$$

- The above equation v, forms the basis for determining the intermediate pixel's position (i.e. deflection voltage in analog device like CRT).
- When the value of m is calculated using equation ii, test for three cases can be performed as:

Case I: For $|m| < 1$

- Δx can be set to small increment (Generally 1) and the corresponding Δy calculated from equation (v). (Δx range is high)

Case II: For $|m| > 1$

- Δy can be set to small increment (Generally 1) and the corresponding Δx calculated from equation (v). (Δy range is high)

Case III: For $|m| = 1$

- $\Delta y = \Delta x$, x and y pixels both are equal.

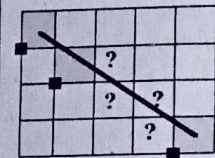
- In each case, a smooth line with slope m is generated between the specified points.

Line Drawing Algorithms

We are going to analyze how this process is achieved.

Some useful definitions

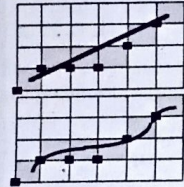
Rasterization: Process of determining which pixels provide the best approximation to a desired line on the screen.



Scan Conversion: Combination of rasterization and generating the picture in scan line order.

General requirements

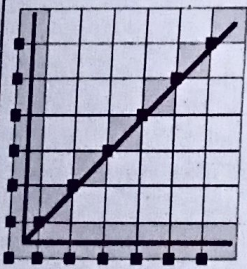
- Straight lines must appear as straight lines.



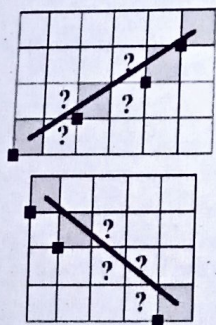
- They must start and end accurately
- Lines should have constant brightness along their length
- Lines should draw rapidly

Line Drawing Algorithms

For horizontal, vertical and 45° lines, the choice of raster elements is obvious. These lines exhibit constant brightness along the length:



For any other orientation the choice is more difficult.



Line Drawing Algorithm

Direct method of line drawing

A straight line can be represented in a slope-intercept equation as:

$$y = mx + b \quad \text{..... (1)}$$

Where,

m is slope of the line and b is y -intercept

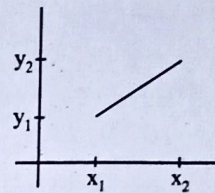


Fig.: Line path between two endpoint positions

A line is drawn by joining two end points, For any two given points (x_1, y_1) and (x_2, y_2) , Slope can be calculated by,

$$m = \frac{y_2 - y_1}{x_2 - x_1} \quad \text{..... (2)}$$

$$b = y_1 - mx_1 \quad \text{..... (3)}$$

At any point (x_k, y_k)

$$y_k = mx_k + b$$

At any point (x_{k+1}, y_{k+1})

$$y_{k+1} = mx_{k+1} + b \quad \text{..... (4)}$$

When the value of m is calculated using equation 2, the following test cases can be performed as,

Case I: For lines with slope magnitude $|m| < 1$, we set small increment in x axis (Δx) equal to 1 and calculate corresponding y increment with the help of equation no. 4. i.e.,

$$x_{\text{next}} = x_{\text{previous}} + 1$$

$$y_{\text{next}} = mx_{\text{next}} + b$$

Case II: For lines with slope magnitude $|m| > 1$, we set small increment in y axis (Δy) equal to 1 and calculate corresponding x increment with the help of equation no. 4.

$$y_{\text{next}} = y_{\text{previous}} + 1$$

$$x_{\text{next}} = \frac{y_{\text{next}} - b}{m}$$

On raster systems, we must sample a line at discrete positions and determine the nearest pixel to the line at each sampled position.

Q. Digitize the line joining the points (1, 2) and (5, 3) with direct method.

Hints;

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{1}{4}, \quad b = y_1 - mx_1 = \frac{7}{4}$$

Here, $m < 1$

So,

$$x_{\text{next}} = x_{\text{previous}} + 1$$

$$y_{\text{next}} = mx_{\text{next}} + b$$

x	y = mx + b	Plotpixel (x, y)
1	2	(1, 2)
2	2.25	(2, 2)
3	2.5	(3, 3)
4	2.75	(4, 3)
5	3	(5, 3)

Q. Digitize the line joining the points (3, 4) and (9, 5) with direct method.

Hints,

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{1}{6}, \quad b = y_1 - mx_1 = \frac{7}{2}$$

Here, $m < 1$

So,

$$x_{\text{next}} = x_{\text{previous}} + 1$$

$$y_{\text{next}} = mx_{\text{next}} + b$$

x	y = mx + b	Plotpixel (x, y)
3	4	(3, 4)
4	4.17	(4, 4)
5	4.33	(5, 4)
6	4.50	(6, 5)
7	4.66	(7, 5)
8	4.83	(8, 5)
9	5	(9, 5)

DDA line Drawing algorithm

The vector representation algorithms (and curve generation algorithms) which step along the line (or curve) to determine the pixels which should be turned on are sometimes called digital differential analyzer (DDA). The name comes from the fact that we use the same technique as a numerical method for solving differential equations.

The equation of the line is given,

$$Y = m \cdot x + b \quad \text{..... (i)}$$

$$m = (y_2 - y_1) / (x_2 - x_1) \quad \text{..... (ii)}$$

for any interval Δx , corresponding interval is given by $\Delta y = m \cdot \Delta x$

Case I:

If $|m| \leq 1$, we sample at unit x interval i.e. $\Delta x = 1$

$$x_{k+1} = x_k + 1 \quad \text{..... (iii)}$$

Then we compute each successive y-values, by setting $\Delta y = m$

(since, $m = \frac{\Delta y}{\Delta x}$ and $\Delta x = 1$)

$$\text{So, } y_{k+1} = y_k + m \quad \text{..... (iv)}$$

The calculated y value must be rounded to the nearest integer.

Case II:

If $|m| > 1$, we sample at unit y-interval i.e. $\Delta y = 1$ and compute each successive x-values.

$$y_{k+1} = y_k + 1 \quad \text{..... (v)}$$

Therefore, $1 = m \cdot \Delta x$, and $\Delta x = \frac{1}{m}$ (since, $m = \frac{\Delta y}{\Delta x}$ and $\Delta y = 1$)

$$x_{k+1} = x_k + \frac{1}{m} \quad \text{..... (vi)}$$

The above equations are under the assumption that lines are to be processed from left endpoints (x_k, y_k) to right endpoints (x_{k+1}, y_{k+1}). If the line is to be processed from right to left i.e. starting end point is at the right then,

$$\Delta x = -1$$

$$\bullet \text{ For } |m| \leq 1, x_{k+1} = x_k - 1 \text{ and } y_{k+1} = y_k - m \quad \text{..... (vii)}$$

And for slope greater than 1, $\Delta x = -1$

$$\bullet \text{ For } |m| > 1, y_{k+1} = y_k - 1 \text{ and } x_{k+1} = x_k - \frac{1}{m} \quad \text{..... (viii)}$$

The obtained equations also can be used to calculate the pixel position along a line with negative slope as below.

- If $|m| < 1$ and the start end point is left, we set $\Delta x = 1$ and calculate the Y value as in case i.
- If $|m| < 1$ and the start end point is right, we set $\Delta x = -1$ and calculate the Y value as in equation vii.
- If $|m| > 1$ and start end point is left, we set $\Delta y = 1$ and calculate the Y value as in case ii.
- If $|m| > 1$ and start end point is right, we set $\Delta y = -1$ and calculate the Y value as in equation vii.

We sample the line at unit interval in one co-ordinate and determine corresponding integer values nearest the path for the other co-ordinate.

Different cases:

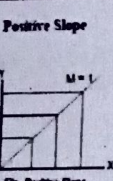
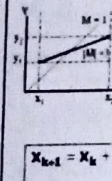
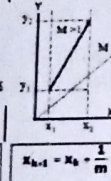
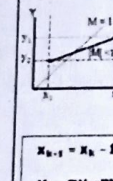
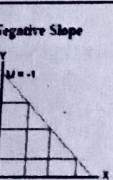
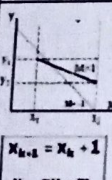
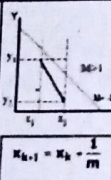
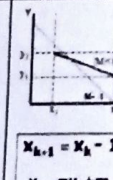
Case 1: Line with positive slope and magnitude less than or equal to 1 ($|m| \leq 1$)

Case 2: Line with positive slope and magnitude more than 1 ($|m| > 1$)

Case 3: Line with negative slope and magnitude less than or equal to 1 ($|m| \leq 1$)

Case 4: Line with negative slope and magnitude more than 1 ($|m| > 1$)

The possible combinations can be shown as follows:

DDA	Moving Left to Right		Moving Right to Left	
	Slope (m) < 1	Slope (m) > 1	Slope (m) < 1	Slope (m) > 1
Positive Slope	 Fig. Positive Slope	 $x_{k+1} = x_k + 1$ $y_{k+1} = y_k + m$	 $x_{k+1} = x_k - 1$ $y_{k+1} = y_k - m$	 $x_{k+1} = x_k - 1$ $y_{k+1} = y_k - \frac{1}{m}$
Negative Slope	 Fig. Negative Slope	 $x_{k+1} = x_k + 1$ $y_{k+1} = y_k - m$	 $x_{k+1} = x_k - 1$ $y_{k+1} = y_k + m$	 $x_{k+1} = x_k - 1$ $y_{k+1} = y_k + \frac{1}{m}$

1. Line with a positive slope

Case I:

If slope is positive with magnitude lesser than or equal to 1, $|m| \leq 1$, then we sample at unit x intervals ($\Delta x = 1$) and compute each

successive y value because the increment in x is more than increment in y.

So, set $\Delta x = 1$.

Now, from equation (I), we get

$$\Delta y = m$$

$$\text{i.e., } x_{k+1} = x_k + \Delta x = x_k + 1 \quad \dots\dots\dots (3)$$

$$y_{k+1} = y_k + \Delta y = y_k + m \quad \dots\dots\dots (4)$$

Where subscript k takes integer values starting from 1, for the first point, and increases by 1 unit until the final end point is reached. Since m can be any real number between 0 to 1, the calculated y values must be rounded to the nearest integer.

Case II:

If slope is positive with magnitude greater than 1, $|m| > 1$, then the increment in x (Δx) is smaller than increment in y (Δy). So, we sample at unit y intervals ($\Delta y = 1$) and calculate each succeeding x value as,

$$\Delta x = \frac{1}{m}$$

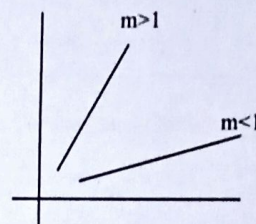


Fig.: Line with positive slope $|m| < 1$ and $|m| > 1$

That is,

$$x_{k+1} = x_k + \frac{1}{m} \quad \dots\dots\dots (5)$$

$$y_{k+1} = y_k + 1 \quad \dots\dots\dots (6)$$

Here, in the both case $m \leq 1$ and $m > 1$, we consider the algorithm (i.e., equations 3, 4, 5, 6) based on the assumption that lines are to be processed from the left end point to the right end point.

If this process is reversed (that is, lines are to be processed from right to left), relation is required to be changed in both case:

Case I:

If sloped is positive with magnitude lesser than or equal to 1, $|m| \leq 1$, then $\Delta x = -1$, so, $\Delta y = -m$

$$\text{i.e., } x_{k+1} = x_k - 1$$

$$y_{k+1} = y_k - m$$

Case II:

If slope is positive with magnitude greater than 1, $|m| > 1$,

then $\Delta y = -1$ and $\Delta x = -\frac{1}{m}$

$$\text{i.e., } x_{k+1} = x_k - \frac{1}{m} \quad \dots\dots\dots (7)$$

$$y_{k+1} = y_k - 1 \quad \dots\dots\dots (8)$$

2. Line with negative slope

Case I:

If slope is negative with magnitude lesser than or equal to 1, $|m| \leq 1$, then assume start end point is the left.

$\Delta x = 1$ and $\Delta y = m$ (m is negative)

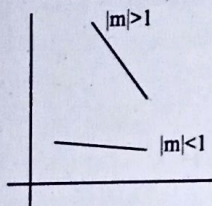


Fig.: Line with negative slope $|m| < 1$ and $|m| > 1$

That is,

$$x_{k+1} = x_k + 1 \quad \dots\dots\dots (9)$$

$$y_{k+1} = y_k + m \quad \dots\dots\dots (10)$$

If algorithm is required to proceed right to left, then set on.

$\Delta x = -1$ and $\Delta y = -m$

$$\text{i.e., } x_{k+1} = x_k - 1 \quad \dots\dots\dots (11)$$

$$y_{k+1} = y_k - m \quad \dots\dots\dots (12)$$

Case II:

If slope is negative with magnitude greater than 1, $|m| > 1$, then assume start end is at left and set $\Delta y = -1$ and $\Delta x = -\frac{1}{m}$

$$\text{i.e., } x_{k+1} = x_k - \frac{1}{m} \quad \dots\dots\dots (13)$$

$$y_{k+1} = y_k - 1 \quad \dots\dots\dots (14)$$

If the, algorithm is required to proceed right to left, then set

$$\Delta y = 1 \text{ and } \Delta x = \frac{1}{m}$$

$$\text{i.e., } x_{k+1} = x_k + \frac{1}{m} \quad \dots\dots\dots (15)$$

$$y_{k+1} = y_k + 1 \quad \dots\dots\dots (16)$$

DDA Algorithm

Step 1: Start Algorithm

Step 2: Declare x_1, y_1, x_2, y_2, dx , step as integer variable and x, y, x_{inc}, y_{inc} as floating point

Step 3: Read the values of two end points x_1, y_1, x_2, y_2

Step 4: Calculate $dx = x_2 - x_1$

Step 5: Calculate $dy = y_2 - y_1$

Step 6: Calculate the value of step

If absolute $(dx) >$ absolute (dy)

Then step = absolute (dx)

Else step = absolute (dy)

Step 7: Select the raster unit

$$x_{inc} = \frac{dx}{step}$$

$$y_{inc} = \frac{dy}{step}$$

Assign $x = x_1$

Assign $y = y_1$

Step 8: Set pixel (x, y)

Step 9: $x = x + x_{inc}$

$y = y + y_{inc}$

Plot pixels (Round (x) , Round (y))

Step 10: Repeat step 9 until $x = x_2$

Step 11: End Algorithm

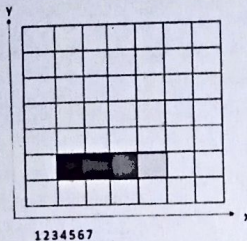
Advantage of DDA

1. It is simple to understand.
2. It is faster method than direct use of line equation $y = mx + c$. (Removes multiplication operation and only involve increments operation of x or y direction)
3. It requires no special skills for implementation

Disadvantages of DDA

1. A floating point addition is still needed in determining each successive point which is time consuming.
2. The accumulation of round off error in successive addition of the floating point increments may cause, the calculated pixel position to drift away from the true line path for long line segment.

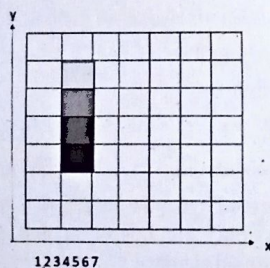
Example: Horizontal line



1. $(X_1, Y_1) = (2, 2)$
 $(X_2, Y_2) = (5, 2)$
 $\Delta x = 3$
 $\Delta y = 0$
 $m = \Delta y / \Delta x = 0$
Steps = 3
 $\Delta x = \Delta x / \text{step} = 1$
 $\Delta y = \Delta y / \text{step} = 0$

x_i	y_i	x_{i+1}	y_{i+1}
2	2	3	2
3	2	4	2
4	2	5	2

Example: Vertical line



1. $(X_1, Y_1) = (2, 3)$
 $(X_2, Y_2) = (2, 6)$

$$\Delta x = 0$$

$$\Delta y = 3$$

$$m = \Delta y / \Delta x = \text{infinite}$$

$$\text{Steps} = 3$$

$$\Delta x = \Delta x / \text{step} = 0$$

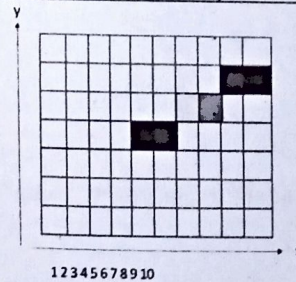
$$\Delta y = \Delta y / \text{step} = 1$$

x_i	y_i	x_{i+1}	y_{i+1}
2	3	2	4
2	4	2	5
2	5	2	6

Example: $m < 1$

1. $(X_1, Y_1) = (5, 4)$
 $(X_2, Y_2) = (10, 6)$
 $\Delta x = 5$
 $\Delta y = 2$
 $m = \Delta y / \Delta x = 0.4$
Steps = 5
 $\Delta x = \Delta x / \text{step} = 1$
 $\Delta y = \Delta y / \text{step} = 2/5 = 0.4$
From DDA algorithm
 $X_{i+1} = X_i + 1$
 $Y_{i+1} = Y_i + m$

x	y	x_{i+1}	y_{i+1}	Y round off	Plot (x, y)
5	4	6	4.4	4	(6,4)
6	4.4	7	4.8	5	(7,5)
7	4.8	8	5.2	5	(8,5)
8	5.2	9	5.6	6	(9,6)
9	5.6	10	6	6	(10,6)



Points

(5,4), (6,4), (7,5), (8,5), (9,6), (10,6)

Example: $m > 1$

1. $(X_1, Y_1) = (5,4)$

$(X_2, Y_2) = (8,9)$

$\Delta x = 3$

$\Delta y = 5$

$m = \Delta y / \Delta x = 1.67$

Steps = 5

$\Delta x = \Delta x / \text{step} = 0.6$

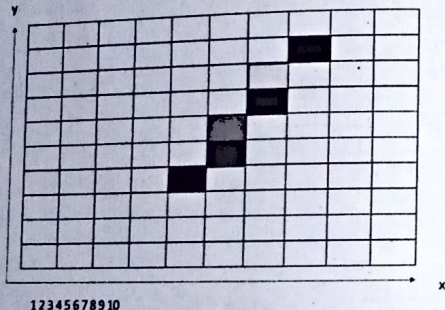
$\Delta y = \Delta y / \text{step} = 5/5 = 1$

From DDA algorithm

$X_{i+1} = X_i + 1/m$

$Y_{i+1} = Y_i + 1$

x	y	X_{i+1}	Y_{i+1}	X round off	Plot (x,y)
5	4	5.6	5	6	(6,5)
5.6	5	6.2	6	6	(6,6)
6.2	6	6.8	7	7	(7,7)
6.8	7	7.4	8	7	(7,8)
7.4	8	8	9	8	(8,9)



Points

(5,4), (6,5), (6,6), (7,7), (7,8), (8,9)

Q. Digitize a Line with end point A(2, 3) and B(6, 8), using DDA.

Here, Slope (M) = $\frac{(8-3)}{(6-2)} = 1.25$

Here Slope is positive and greater than 1

And moving left to right and $\frac{1}{m} = 0.8$

So, we use $X_{k+1} = X_k + \frac{1}{M}$

$Y_{k+1} = Y_k + 1$

K	$X_{k+1} = X_k + \frac{1}{M}$	$Y_{k+1} = Y_k + 1$	(X, Y)
1	$= 2 + 0.8 = 2.8 \sim 3$	4	(3,4)
2	$= 2.8 + 0.8 = 3.6 \sim 4$	5	(4,5)
3	$= 3.6 + 0.8 = 4.4 \sim 4$	6	(4,6)
4	$= 4.4 + 0.8 = 5.2 \sim 5$	7	(5,7)
5	$= 5.2 + 0.8 = 6$	8	(6,8)

Solved Numerical Problems

- Consider a line from (2,1) to (8,3). Using simple DDA algorithm, rasterize this line.

Solution:

Given Starting Point: $(x_1, y_1) = (2,1)$ and

Ending Point: $(x_2, y_2) = (8,3)$

Now Slope $m = (y_2 - y_1) / (x_2 - x_1) = (3-1) / (8-2) = (1/3) = 0.333$

Since $|m| < 1$, From DDA algorithm we have

$X_{k+1} = X_k + 1$

$Y_{k+1} = Y_k + (M)$

X	Y	X-Plot	Y-Plot	(X,Y)
2	1	2	1	(2,1)
3	1.33	3	1	(3,1)
4	1.66	4	2	(4,2)
5	1.993	5	2	(5,2)
6	2.326	6	2	(6,2)
7	2.659	7	3	(7,3)
8	2.999	8	3	(8,3)

2. Digitized the line with end points (0,0) and (4,5) using DDA.

Solution:

Given Starting Point: $(x_1, y_1) = (0,0)$ and Ending Point $(x_2, y_2) = (4,5)$

Now Slope $m = (y_2 - y_1) / (x_2 - x_1) = (5-0) / (4-0) = 5/4 = 1.25$

Since $|m| > 1$, From DDA algorithm we have

$$y_{k+1} = y_k + 1$$

$$x_{k+1} = x_k + (1/m)$$

X	Y	X-Plot	Y-Plot	(X,Y)
0	0	0	0	(0,0)
0.8	1	1	1	(1,1)
1.6	2	2	2	(2,2)
2.4	3	2	3	(2,3)
3.2	4	3	4	(3,4)
4	5	4	5	(4,5)

3. Digitized the line with end points (3,7) and (8,3) using DDA.

Solution:

Given Starting Point: $(x_1, y_1) = (3,7)$ and Ending Point: $(x_2, y_2) = (8,3)$

Now Slope $m = (y_2 - y_1) / (x_2 - x_1) = (3-7) / (8-3) = (-4/5) = -0.8$

Since $|m| < 1$, From DDA algorithm we have

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k + (M)$$

X	Y	X-Plot	Y-Plot	(X,Y)
3	7	3	7	(3,7)
4	$7 - 0.8 = 6.2$	4	6	(4,6)
5	5.4	5	5	(5,5)
6	4.6	6	5	(6,5)
7	3.8	7	4	(7,4)
8	3	8	3	(8,3)

4. Draw line using DDA (3,7) (8,3)

Solution:

$$\Delta x = x_2 - x_1 = 5$$

$$\Delta y = y_2 - y_1 = -4$$

$$\text{Steps} = 5$$

$$m = \frac{-4}{5}$$

$$x_{inc} = \frac{\Delta x}{5} = \frac{5}{5} = 1$$

$$y_{inc} = \frac{-4}{5} = -0.8$$

x_i	y_i	x_{i+1}	y_{i+1}	y_{round}	$x_{plot} y_{plot}$
3	7	4	6.2	6	(4,6)
4	6.2	5	5.4	5	(5,5)
5	5.4	6	4.6	5	(6,5)
6	4.6	7	3.8	4	(7,4)
7	3.8	8	3	3	(8,3)

5. Draw line using (5,5) (0,0) using DDA

Solution:

$$\Delta x = 0 - 5 = -5$$

$$\Delta y = 0 - 5 = -5$$

$$m = \frac{\Delta y}{\Delta x} = 1$$

$$\text{Steps} = 5$$

$$\Delta x = \frac{\Delta x}{5} = \frac{-5}{5} = -1$$

$$\Delta y = \frac{\Delta y}{5} = -1$$

x_i	y_i	x_{i+1}	y_{i+1}	Plot
5	5	4	4	4,4
4	4	3	3	3,3
3	3	2	2	2,2
2	2	1	1	1,1
1	1	0	0	0,0

6. Draw line using (3,6) - (5,9)

Solution:

$$\Delta x = 2$$

$$\Delta y = 3$$

$$m = \frac{3}{2} > 1$$

$$X_{i+1} = X_i + \frac{1}{m}$$

$$Y_{i+1} = Y_i + 1$$

Step = 3

$$X_{inc} = \frac{2}{3} = 0.67$$

$$Y_{inc} = \frac{3}{3} = 1$$

X_i	Y_i	X_{i+1}	Y_{i+1}	X_{round}	Plot
3	6	3.67	7	4	4,7
3.67	7	4.34	8	4	4,8
4.34	8	5.01	9	5	5,9

7. Consider a line from (0,0) to (5,5). Using simple DDA algorithm rasterize this line.

Solution:

Given starting point: $(x_1, y_1) = (0,0)$ and

Ending point: $(x_2, y_2) = (5,5)$

Now Slope $m = (y_2 - y_1) / (x_2 - x_1)$

$$= (5 - 0) / (5 - 0) = (5/5) = 1$$

Since $|m| \leq 1$, From DDA algorithm we have

$$X_{k+1} = X_k + 1$$

$$Y_{k+1} = Y_k + (M)$$

X	Y	X-Plot	Y-Plot	(X,Y)
0	0	0	0	(0,0)
1	1	1	1	(1,1)
2	2	2	2	(2,2)
3	3	3	3	(3,3)
4	4	4	4	(4,4)
5	5	5	5	(5,5)

Bresenham's Algorithm

The algorithm selects the best pixel co-ordinate by testing the sign of integer parameter whose value is proportional to the difference between the separations of two-pixel actual line path.

1. Line with positive slope and magnitude $|m| < 1$

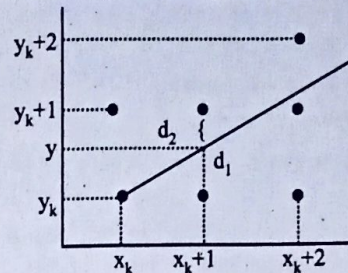


Fig.: Line with $m < 1$

- Pixel positions are determined by sampling at unit x intervals.
- Starting from left end position (x_0, y_0) of a given line, we step to each successive column (x-position) and plot the pixel whose scan line y value is closed to the line path.

Assuming the pixel at (x_k, y_k) to be displayed is determined, we next to decide which pixel to plot in column x_{k+1} , our choices are the pixels at positions.

$$(x_k + 1, y_k) \text{ and } (x_k + 1, y_k + 1)$$

At sampling position $x_k + 1$, we label vertical pixel separations from the mathematical line path d_1 and d_2 .

The y-co-ordinate the mathematical at pixel column position $x_k + 1$ is calculation.

As,

$$y = m(x_k + 1) + b \dots\dots\dots (1)$$

Then,

$$d_1 = y - y_k$$

$$d_1 = m(x_k + 1) + b - y_k \dots\dots\dots (2)$$

And,

$$d_2 = (y_k + 1) - y$$

$$d_2 = y_k + 1 - m(x_k + 1) - b \dots\dots\dots (3)$$

Now,

$$d_1 - d_2 = m(x_k + 1) + b - y_k - y_k - 1 + m(x_k + 1) + b$$

$$= 2m(x_k + 1) - 2y_k + 2b - 1$$

$$= 2 \frac{\Delta y}{\Delta x} (x_k + 1) - 2y_k + 2b - 1$$

Defining decision parameter $p_k = \Delta x(d_1 - d_2)$

$$\begin{aligned}
 p_k &= \Delta x(d_1 - d_2) = 2\Delta y(x_k + 1) - 2\Delta xy_k + 2\Delta xb - \Delta x \\
 &= 2\Delta yx_k + 2\Delta y - 2\Delta xy_k + \Delta x(2b - 1) \\
 &= 2\Delta yx_k - 2\Delta xy_k + 2\Delta y + \Delta x(2b - 1) \\
 &= 2\Delta yx_k - 2\Delta xy_k + c \dots\dots\dots (4)
 \end{aligned}$$

Where, $c = 2\Delta y + \Delta x(2b - 1)$
 Here, sign of p_k is same, as the sign of $d_1 - d_2$ since $\Delta x > 0$.

Case I:

If $p_k < 0$ then $d_1 < d_2$ which implies pixel at y_k is nearer than pixel at $(y_k + 1)$. So, pixel at y_k is better to choose which reduce error than pixel at $(y_k + 1)$. This determines next pixel co-ordinate to plot is $(x_k + 1, y_k)$

Case II:

If $p_k \geq 0$, then $d_2 < d_1$ which implies that $y_k + 1$ is nearer than y_k . So, pixel at $(y_k + 1)$ is better to choose which reduce error than pixel at y_k . This determines next pixel co-ordinate to plot is $(x_k + 1, y_k + 1)$.

Now, similarly, pixel as $(x_k + 2)$ can be determined whether it is $(x_k + 2, y_k + 1)$ or $(x_k + 2, y_k + 2)$ by looking the sign of deciding parameter p_{k+1} assuming pixel as $(x_k + 1)$ is known.

$$p_{k+1} = 2\Delta yx_{k+1} - 2\Delta xy_{k+1} + c \text{ where } c \text{ is same as in } p_k$$

Now,

$$\begin{aligned}
 p_{k+1} - p_k &= 2\Delta yx_{k+1} - 2\Delta xy_{k+1} + c - (2\Delta yx_k - 2\Delta xy_k + c) \\
 &= 2\Delta y(x_{k+1} - x_k) - 2\Delta x(y_{k+1} - y_k) \text{ where } x_{k+1} = x_k + 1 \\
 &= 2\Delta y(x_k + 1 - x_k) - 2\Delta x(y_{k+1} - y_k) \\
 &= 2\Delta y - 2\Delta x(y_{k+1} - y_k)
 \end{aligned}$$

This implies that decision parameter for the current column can be determined if the decision parameter of the last column is known.

Here, $(y_{k+1} - y_k)$ could either 0 or 1 which depends on sign of p_k .

Case I:

If $p_k < 0$ (i.e., $d_1 < d_2$),

$y_{k+1} = y_k$ which implies $(y_{k+1} - y_k = 0)$

i.e., at $p_k < 0$, then pixel to be plotted is (x_{k+1}, y_k)

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k$$

and,

$$p_{k+1} = p_k + 2\Delta y$$

Case II:

If $p_k \geq 0$ (i.e., $d_2 < d_1$), $y_{k+1} = y_k + 1$ which implies $(y_k + 1 - y_k) = 1$

That is, at $p_k \geq 0$,

the pixel to plot is $(x_k + 1, y_k + 1)$

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k + 1$$

and,

$$p_{k+1} = p_k + 2\Delta y - 2\Delta x$$

Initial decision parameter (p_0)

Here, $p_k = 2\Delta yx_k - 2\Delta xy_k + 2\Delta y + \Delta x(2b - 1)$

$$p_0 = 2\Delta yx_0 - 2\Delta xy_0 + 2\Delta y + \Delta x(2b - 1)$$

$$\text{But } b = y_0 - mx_0 = y_0 - \frac{\Delta y}{\Delta x} x_0$$

$$p_0 = 2\Delta yx_0 - 2\Delta xy_0 + 2\Delta y + 2\Delta xy_0 - 2\Delta yx_0 - \Delta x = 2\Delta y - \Delta x$$

2. Line with positive slope and magnitude $|m| > 1$

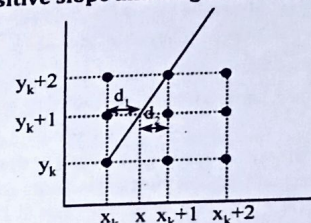


Fig.: Line with $m > 1$

- Pixel positions are determined by sampling at unit y intervals.
- Starting from left end position (x_0, y_0) of a given line, we step to each successive row (y -position) and plot the pixel whose scan line x value is closed to the line path.

Assuming the pixel at (x_k, y_k) to be displayed is determined, we next to decide which pixel to plot in row y_{k+1} , our choices are the pixels at position.

$(x_k, y_k + 1)$ and $(x_k + 1, y_k + 1)$

At sampling position $y_k + 1$, we label horizontal pixel separations from the mathematical line path, d_1 and d_2 .

The x -co-ordinate on the mathematical at pixel row position y_{k+1} is calculation.

As,

$$y_k + 1 = mx + b$$

$$x = (y_k + 1 - b)/m \dots\dots\dots(1)$$

Then,

$$d_1 = x - x_k \dots\dots\dots(2)$$

And,

$$d_2 = (x_k + 1) - x \dots\dots\dots(3)$$

Now,

$$d_1 - d_2 = x - x_k - x_k - 1 + x$$

$$= 2x - 2x_k - 1$$

$$x = (y_k + 1 - b)/m$$

$$d_1 - d_2 = 2(y_k + 1 - b)/m - 2x_k - 1$$

$$= 2\Delta x(y_k + b)/\Delta y - 2x_k - 1$$

$$\Delta y(d_1 - d_2) = 2\Delta xy_k + 2\Delta x - 2\Delta xb - 2\Delta yx_k - \Delta y$$

Defining decision parameter $p_k = \Delta y(d_1 - d_2)$

$$p_k = \Delta y(d_1 - d_2) = 2\Delta xy_k + 2\Delta x - 2\Delta xb - 2\Delta yx_k - \Delta y$$

$$= 2\Delta xy_k - 2\Delta yx_k + c \dots\dots\dots(4)$$

Where; $c = 2\Delta x - 2\Delta xb - \Delta y$

Here, sign of p_k is same as the sign of $d_1 - d_2$ since $\Delta y > 0$.

Case I:

If $p_k < 0$ then $d_1 < d_2$ which implies pixel at x_k is nearer than pixel at $(x_k + 1)$. So, pixel at x_k is better to choose which reduce error than pixel at $(x_k + 1)$. This determines next pixel co-ordinate to plot is $(x_k, y_k + 1)$

Case II:

If $p_k \geq 0$, then $d_1 > d_2$, which implies that $x_k + 1$ is nearer than x_k . So, pixel at $(x_k + 1)$ is better to choose which reduce error than pixel at x_k . This determines next pixel co-ordinate to plot is $(x_k + 1, y_k + 1)$.

Now, similarly, pixel as $(y_k + 2)$ can be determined whether it is $(x_k + 1, y_k + 2)$ or $(x_k + 2, y_k + 2)$ by looking the sign of deciding parameter p_{k+1} assuming pixel as $(y_k + 1)$ is known.

$$p_{k+1} = 2\Delta x y_{k+1} - 2\Delta y x_{k+1} + c \text{ where } c \text{ is same as in } p_k$$

Now,

$$p_{k+1} - p_k = 2\Delta x y_{k+1} - 2\Delta y x_{k+1} + c - 2\Delta x y_k + 2\Delta y x_k - c$$

$$= 2\Delta x(y_{k+1} - y_k) - 2\Delta y(x_{k+1} - x_k)$$

$$= 2\Delta x(y_{k+1} - y_k) - 2\Delta y(x_{k+1} - x_k)$$

$$= 2\Delta x(y_k + 1 - y_k) - 2\Delta y(x_{k+1} - x_k)$$

$$= 2\Delta x - 2\Delta y(x_{k+1} - x_k)$$

This implies that decision parameter for the current row can be determined if the decision parameter of the last row is known. Here, $(x_{k+1} - x_k)$ could either 0 or 1 which depends on sign of p_k .

Case I:

If $p_k < 0$ (i.e., $d_1 < d_2$), then $x_{k+1} = x_k$ which implies $(x_{k+1} - x_k = 0)$ i.e., at $p_k < 0$, then pixel to be plotted is $(x_k, y_k + 1)$

$$x_{k+1} = x_k$$

$$y_{k+1} = y_k + 1$$

and,

$$p_{k+1} = p_k + 2\Delta x$$

Case II:

If $p_k \geq 0$ (i.e., $d_1 > d_2$), then $x_{k+1} = x_k + 1$ which implies $(x_{k+1} - x_k = 1)$

That is, at $p_k \geq 0$, the pixel to plot is $(x_k + 1, y_k + 1)$

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k + 1$$

and,

$$p_{k+1} = p_k + 2\Delta x - 2\Delta y$$

Initial decision parameter (p_0)

$$p_k = 2\Delta x y_k + 2\Delta x - 2\Delta x b - 2\Delta y x_k - \Delta y$$

$$p_0 = 2\Delta x y_0 + 2\Delta x - 2\Delta x b - 2\Delta y x_0 - \Delta y$$

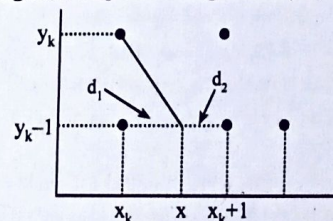
$$b = y_0 - mx_0$$

$$p_0 = 2\Delta x y_0 + 2\Delta x - 2\Delta x(y_0 - \Delta y x_0/\Delta x) - 2\Delta y x_0 - \Delta y$$

$$= 2\Delta x y_0 + 2\Delta x - 2\Delta x y_0 - 2\Delta x \times \Delta y x_0/\Delta x - 2\Delta y x_0 - \Delta y$$

$$= 2\Delta x - \Delta y$$

3. Line with negative slope and magnitude $|m| > 1$



- Pixel positions are determined by sampling at unity intervals.
- Starting from left end position (x_0, y_0) of a given line, we step to each successive row (y position) and plot the pixel whose x value is closest to the line path.
- Assuming the pixel at (x_k, y_k) to be displayed is determined, we next need to decide which pixel to plot in row $y_k - 1$.
- The candidate pixels are at position $(x_k, y_k - 1)$ and $(x_k + 1, y_k - 1)$.
- At sampling position $y_k - 1$, we label horizontal pixel separation from the mathematical line path d_1 and d_2 .
- The x -coordinate on the mathematical line at pixel row position $y_k - 1$ is calculated as

$$y_k - 1 = mx + b$$

$$\text{or, } x = \frac{y_k - 1 - b}{m}$$

where $m = -\frac{\Delta y}{\Delta x}$ since m is negative.

Then,

$$d_1 = x - x_k \dots\dots\dots(ii)$$

$$d_2 = x_k + 1 - x \dots\dots\dots(iii)$$

And,

$$d_1 - d_2 = x - x_k - x_k - 1 + x$$

$$= 2x - 2x_k - 1$$

$$= 2\left(\frac{y_k - 1 - b}{m}\right) - 2x_k - 1$$

$$= 2\left(-\frac{\Delta x}{\Delta y}\right)(y_k - 1 - b) - 2x_k - 1$$

$$(d_1 - d_2)\Delta y = -2\Delta x(y_k - 1 - b) - (2x_k + 1)\Delta y$$

Now, defining decision parameter $p_k = \Delta y (d_1 - d_2)$

$$p_k = \Delta y(d_1 - d_2) = -2\Delta x(y_k - 1 - b) - (2x_k + 1)\Delta y$$

$$= -2\Delta x y_k + 2\Delta x + 2\Delta x b - 2\Delta y x_k - \Delta y$$

$$= -2\Delta x y_k - 2\Delta y x_k + 2\Delta x + 2\Delta x b - \Delta y$$

$$\therefore p_k = -2\Delta x y_k - 2\Delta y x_k + c \dots\dots\dots(iv)$$

Where $C = 2\Delta x + 2\Delta x b - \Delta y$ (All are constant)

Here, the sign of p_k is same as the sign of $d_1 - d_2$, as $\Delta y > 0$.

Case I:

If $p_k < 0$, then $d_1 < d_2$ which implies that x_k is nearer than $x_k + 1$, so the next pixel to select is $(x_k, y_k - 1)$

Case II:

If $p_k \geq 0$, then $d_1 > d_2$ which implies that $x_k + 1$ is nearer than x_k , so the next pixel to choose is $(x_k + 1, y_k - 1)$

Now,

Similarly, pixel at $(y_k - 2)$ can be determined whether it is $(x_k + 1, y_k - 2)$ or $(x_k + 2, y_k - 2)$ by looking the sign of deciding parameter p_{k+1} assuming pixel at $(y_k - 1)$ is known.

$$p_{k+1} = -2\Delta x y_{k+1} - 2\Delta y x_{k+1} + C$$

where C is same as in p_k .

Now,

$$p_{k+1} - p_k = -2\Delta x y_{k+1} + C + 2\Delta x y_k + 2\Delta y x_k - C$$

$$= -2\Delta x(y_{k+1} - y_k) - 2\Delta y(x_{k+1} - x_k)$$

$$= -2\Delta x(y_k - 1 - y_k) - 2\Delta y(x_{k+1} - x_k)$$

$$= 2\Delta x - 2\Delta y(x_{k+1} - x_k)$$

This implies that decision parameter for the current row can be determined if the decision parameter of the last row is known.

Here $(x_k + 1 - x_k)$ could either 0 or 1 which depends on sign of p_k .

Case I:

If $p_k < 0$, then

$$x_{k+1} = x_k$$

$$y_{k+1} = y_k - 1$$

$$p_{k+1} = p_k + 2\Delta x$$

Case II:

If $p_k \geq 0$ (i.e., $d_1 > d_2$) then

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k - 1$$

$$p_{k+1} = p_k + 2\Delta x - 2\Delta y$$

Now, Initial decision parameter

$$p_k = -2\Delta x y_k - 2\Delta y x_k + 2\Delta x b - 2\Delta x - \Delta y$$

Now,

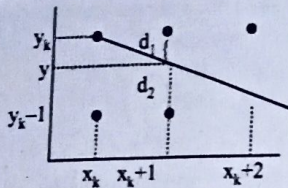
$$p_0 = -2\Delta x y_0 - 2\Delta y x_0 + 2\Delta x b - 2\Delta x - \Delta y$$

$$= -2\Delta x y_0 - 2\Delta y x_0 + 2\Delta x \left(y_0 + \frac{\Delta y}{\Delta x} x_0\right) + 2\Delta x - \Delta y$$

$$\left[y = mx + b, b = y - mx, m = -\frac{\Delta y}{\Delta x} \right]$$

$$\begin{aligned}
 &= 2\Delta xy_0 - 2\Delta yx_0 + \Delta x \left(\frac{2\Delta xy_0 + 2\Delta yx_0}{\Delta x} \right) + 2\Delta x - \Delta y \\
 &= 2\Delta xy_0 - 2\Delta yx_0 + 2\Delta xy_0 + 2\Delta yx_0 + 2\Delta x - \Delta y \\
 &= 2\Delta x - \Delta y
 \end{aligned}$$

4. Line with negative slope and magnitude $|m| < 1$.



$$\begin{aligned}
 d_1 &= y_k - y \\
 d_2 &= y - (y_k - 1) = y - y_k + 1 \\
 d_1 - d_2 &= y_k - y - y + y_k - 1 = 2y_k - 2y - 1
 \end{aligned}$$

Again, $y = m(x_k + 1) + b$

$$d_1 - d_2 = 2y_k - 2(m(x_k + 1) + b) - 1$$

$$m = -\frac{\Delta y}{\Delta x} \text{ (m is negative)}$$

$$d_1 - d_2 = 2y_k - 2\left(-\frac{\Delta y}{\Delta x}\right)(x_k + 1) - 2b - 1$$

$$= \frac{2y_k + \Delta x + 2\Delta yx_k + 2\Delta y - \Delta x(2b) - \Delta x}{\Delta x}$$

$$p_k = (d_1 - d_2) \Delta x = 2\Delta xy_k + 2\Delta yx_k + 2\Delta y - \Delta x(2b) - \Delta x$$

$$p_k = 2\Delta xy_k + 2\Delta yx_k + c$$

$$\text{where } c = 2\Delta y - \Delta x(2b) - \Delta x$$

For next decision parameter,

$$p_{k+1} = 2\Delta xy_{k+1} + 2\Delta yx_{k+1} + c$$

Now,

$$p_{k+1} - p_k = 2\Delta x(y_{k+1} - y_k) + 2\Delta y(x_{k+1} - x_k)$$

Case I:

If $p_k < 0$, $d_1 < d_2$ then

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k$$

$$p_{k+1} = p_k + 2\Delta y$$

Case II:

If $p_k \geq 0$, $d_1 \geq d_2$ then

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k - 1$$

$$p_{k+1} = p_k + 2\Delta y - 2\Delta x$$

Now, initial decision parameter,

$$p_0 = p_0, x_0 = x_0, y_0 = y_0$$

$$p_0 = 2\Delta xy_0 + 2\Delta yx_0 + 2\Delta y - \Delta x \left(2 \left(y_0 - \left(-\frac{\Delta y}{\Delta x} \right) x_0 \right) + 1 \right)$$

$$= 2\Delta xy_0 + 2\Delta yx_0 + 2\Delta y - \Delta x \left(\frac{2\Delta xy_0 + 2\Delta yx_0 + \Delta x}{\Delta x} \right)$$

$$= 2\Delta xy_0 + 2\Delta yx_0 + 2\Delta y - 2\Delta xy_0 - 2\Delta yx_0 - \Delta x$$

$$= 2\Delta y - \Delta x$$

BLA Algorithm

Step 1: Start

Step 2: Declare variables $x_1, y_1, x_2, y_2, l_x, l_y, \Delta x, \Delta y, p_0, p_k, p_{k+1}$

Step 3: Read values of x_1, y_1, x_2, y_2

Step 4: Calculate $\Delta x = \text{absolute}(x_2 - x_1)$

$$\Delta y = \text{absolute}(y_2 - y_1)$$

Step 5: If $(x_2 > x_1)$

assign $l_x = 1$

else

assign $l_x = -1$

Step 6: If $(y_2 > y_1)$

assign $l_y = 1$

else

assign $l_y = -1$

Step 7: Plot (x_1, y_1)

Step 8: If $\Delta x > \Delta y$ (i.e., $|m| < 1$)

$$\text{compute } p_0 = 2\Delta y - \Delta x$$

starting at $k = 0$ to Δx times, repeat

if $(p_k < 0)$

$$x_{k+1} = x_k + l_x$$

$$y_{k+1} = y_k$$

$$p_{k+1} = p_k + 2\Delta y$$

else

$$x_{k+1} = x_k + I_x$$

$$y_{k+1} = y_k + I_y$$

$$p_{k+1} = p_k + 2\Delta y - 2\Delta x$$

plot(x_{k+1}, y_{k+1})

else

$$\text{calculate } p = 2\Delta x - \Delta y$$

starting at $k = 0$ to Δy times, repeat

If ($p_k < 0$)

$$x_{k+1} = x_k$$

$$y_{k+1} = y_k + I_y$$

$$p_{k+1} = p_k + 2\Delta x$$

else

$$x_{k+1} = x_k + I_x$$

$$y_{k+1} = y_k + I_y$$

$$p_{k+1} = p_k + 2\Delta x - 2\Delta y$$

Plot (x_{k+1}, y_{k+1})

Step 9: Stop

Advantage

1. DDA includes calculations related to m and $1/m$ which is still complicated. BLA improves DDA algorithm by only involving integer calculations making it simple and easy to use.
2. Using integer addition and subtraction and multiplication by 2 makes the algorithm more time efficient as addition, subtraction and multiplication operation is computed in lesser time in comparison to division as compared to DDA.
3. BLA is more accurate as compared to DDA because it avoids the ROUND OFF operation and scan converts the line using only incremental integer calculations.

DDA vs BLA

Criteria	DDA Algorithm	Bresenham's Line Algorithm
Arithmetic Operations	Involves floating-point arithmetic	Uses only integer arithmetic
Slope Calculation	Requires calculation of slopes	Does not require slope calculations
Efficiency	Less efficient	More efficient

Criteria	DDA Algorithm	Bresenham's Line Algorithm
Line Restrictions	Handles lines with any slope	Handles lines with slopes between 0 and 1
Line Smoothness	Provides smoother lines	May produce a staircase effect on diagonal lines
Rounding Errors	Prone to rounding errors	Minimizes rounding errors
Computational Steps	May require more steps	Requires fewer steps
Handling Negative Slopes	Handles negative slopes	Requires additional modifications to handle negative slopes
Line Length Limitations	No specific limitations	May have limitations due to integer calculations
Application Flexibility	Suitable for various line types	Primarily used for simple lines (e.g., horizontal, vertical)
Implementation Complexity	Relatively simpler	Requires additional calculations for decision-making
Drawing	Can't draw curves and circles as accurate as BLA	Draw Accurately

Solved Numerical Problems

1. Apply Bresenham's algorithm to draw a line from (20,15) and end point is (30,30)

Solution:

Starting Point (x_0, y_0) = (20,15) Ending Point (x_n, y_n) = (30,30) such that scanning takes place from left to right Slope $m = \frac{(30 - 15)}{(30 - 20)} = 1.5$

i.e. $|M| > 1$

Here:

$$\Delta x = 30 - 20 = 10$$

$$\Delta y = 30 - 15 = 15$$

$$2\Delta x = 2 * 10 = 20$$

$$2\Delta y = 2 * 15 = 30$$

Now from Bresenham's Algorithm for $|M| > 1$, and scanning from left to right

Initial Decision Parameter: $p_0 = p_0 = 2\Delta x - \Delta y$

If $(p_k < 0)$ then

$X_{k+1} = X_k$ and

$Y_{k+1} = Y_k + 1$ and

$P_{k+1} = P_k + 2\Delta x$

otherwise

$X_{k+1} = X_{k+1}$ and

$Y_{k+1} = Y_k + 1$ and

$P_{k+1} = P_k + 2\Delta x - 2\Delta y$

Repeat Δy times

K	P_k	X_{k+1}, Y_{k+1}
0	5	(21,16)
1	-5	(21,17)
2	15	(22,18)
3	5	(23,19)
4	-5	(23,20)
5	15	(24,21)
6	5	(25,22)
7	-5	(25,23)
8	15	(26,24)
9	5	(27,25)
10	-5	(27,26)
11	15	(28,27)
12	5	(29,28)
13	-5	(29,29)
14	15	(30,30)

Therefore the digitized coordinates are $(x_0, y_0) = (20, 15)$, $(x_1, y_1) = (21, 16)$, ..., $(x_{15}, y_{15}) = (30, 30)$

2. Apply Bresenham's algorithm to draw a line from $(-3, 0)$ and end point is $(4, 4)$

Solution:

Starting Point $(x_0, y_0) = (-3, 0)$ Ending Point $(x_n, y_n) = (4, 4)$ such that

scanning takes place from left to right Slope $m = \frac{(4 - 0)}{(4 + 3)} = \frac{4}{7} = 0.57$

i.e. $|M| < 1$

Here:

$$\Delta x = 4 + 3 = 7$$

$$\Delta y = 4 - 0 = 4$$

$$2\Delta x = 2 * 7 = 14$$

$$2\Delta y = 2 * 4 = 8$$

Now from Bresenham's Algorithm for $|M| < 1$, and scanning from left to right

Initial Decision Parameter: $P_0 = 2\Delta y - \Delta x$

If $(p_x < 0)$ then

$X_{k+1} = X_k + 1$ and

$Y_{k+1} = Y_k$ and

$P_{k+1} = P_k + 2\Delta y$

otherwise

$X_{k+1} = X_k + 1$ and

$Y_{k+1} = Y_{k+1}$ and

$P_{k+1} = P_k + 2\Delta y - 2\Delta x$

Repeat Δx times

K	P_k	X_{k+1}, Y_{k+1}
0	1	(-2,1)
1	-5	(-1,1)
2	3	(0,2)
3	-3	(1,2)
4	5	(2,3)
5	-1	(3,3)
6	7	(4,4)

Therefore the digitized coordinates are $(x_0, y_0) = (-3, 0)$, $(x_1, y_1) = (-2, 1)$, ..., $(x_7, y_7) = (4, 4)$

Note: If the initial point for a line with positive slope is right endpoint, then both X and Y decreases but for negative slope, the procedure is similar, except that now one coordinate decreases as the other increases.

3. Digitize the line with end points $(20, 10)$ and $(30, 18)$ using B.L.A.

Solution:

Here, Starting point of line = $(x_1, y_1) = (20, 10)$ and

Ending point of line = $(x_2, y_2) = (30, 18)$

$$\begin{aligned}\text{Thus, slope of line, } m &= \frac{\Delta y}{\Delta x} = \frac{y_2 - y_1}{x_2 - x_1} \\ &= \frac{(18 - 10)}{(30 - 20)} \\ &= \frac{8}{10}\end{aligned}$$

As the given points, it is clear that the line is moving left to right with the positive slope

$$|m| = 0.8 < 1$$

The initial decision parameter $(P_0) = 2\Delta y - \Delta x = 2 \times 8 - 10 = 6$

Since, for the Bresenham's line drawing Algorithm of slope, $|m| \leq 1$, we have

If $P_k < 0$ (i.e. $d_1 - d_2$ is Negative)

Then,

$$X_{k+1} = X_k + 1$$

$$Y_{k+1} = Y_k$$

$$P_k = P_k + 2\Delta y$$

If $P_k \geq 0$

Then,

$$X_{k+1} = X_k + 1$$

$$Y_{k+1} = Y_k + 1$$

$$P_k = P_k + 2\Delta y - 2\Delta x$$

(20,10)

k	P_k	X_{k+1}	Y_{k+1}	(X_{k+1}, Y_{k+1})
0	6	$20 + 1 = 21$	11	(21, 11)
1	$6 + 2 \times 8 - 2 \times 10 = 2$	$21 + 1 = 22$	12	(22, 12)
2	$2 + 2 \times 8 - 2 \times 10 = -2$	$22 + 1 = 23$	12	(23, 12)
3	$-2 + 2 \times 8 = 14$	$23 + 1 = 24$	13	(24, 13)
4	$14 + 2 \times 8 - 2 \times 10 = 10$	$24 + 1 = 25$	14	(25, 14)
5	$10 + 2 \times 8 - 2 \times 10 = 6$	$25 + 1 = 26$	15	(26, 15)
6	$6 + 2 \times 8 - 2 \times 10 = 2$	$26 + 1 = 27$	16	(27, 16)
7	$2 + 2 \times 8 - 2 \times 10 = -2$	$27 + 1 = 28$	16	(28, 16)
8	$-2 + 2 \times 8 = 14$	$28 + 1 = 29$	17	(29, 17)
9	$14 + 2 \times 8 - 2 \times 10 = 10$	$29 + 1 = 30$	18	(30, 18)

4. Digitize the line with end points (1,0) and (3,3) using BLA.

Solution:

Here, Starting point of line = $(x_1, y_1) = (1, 0)$ and

Ending point of line = $(x_2, y_2) = (3, 3)$

Thus,

$$\begin{aligned}\text{Slope of line, } m &= \frac{\Delta y}{\Delta x} = \frac{y_2 - y_1}{x_2 - x_1} \\ &= \frac{(3 - 0)}{(3 - 1)} \\ &= \frac{3}{2}\end{aligned}$$

As the given points, it is clear that the line is moving left to right with the positive slope,

$$|m| = 1.5 > 1$$

Thus, the initial decision parameter

$$(P_0) = 2\Delta x - \Delta y = 2 \times 2 - 3 = 1$$

$$\Delta x = x_2 - x_1 = 3 - 1 = 2$$

$$\Delta y = y_2 - y_1 = 3 - 0 = 3$$

we have

If $P_k < 0$

Then,

$$X_{k+1} = X_k$$

$$Y_{k+1} = Y_k + 1$$

$$P_k = P_k + 2\Delta x$$

If $P_k \geq 0$

Then,

$$X_{k+1} = X_k + 1$$

$$Y_{k+1} = Y_k + 1$$

$$P_k = P_k + 2\Delta x - 2\Delta y$$

k	P_k	X_{k+1}	Y_{k+1}	(X_{k+1}, Y_{k+1})
0	1	2	1	(2, 1)
1	$1 + 2 \times 2 - 2 \times 3 = -1$	2	$1 + 1 = 2$	(2, 2)
2	$-1 + 2 \times 2 = 3$	3	$2 + 1 = 3$	(3, 3)

5. Draw a line for (10,7) and (5,10) using BLA

Solution:

Here, $(X_1, Y_1) = (10, 7)$ & $(X_2, Y_2) = (5, 10)$

$$M = \frac{(10-7)}{(5-10)} = \frac{-3}{5} \text{ -ve slope and } |M| < 1$$

$$\Delta x = |5-10| = 5$$

$$\Delta y = |10-7| = 3$$

Here,

$$\begin{aligned} \text{Initial } (P_0) &= 2\Delta y - \Delta x \\ &= 2 \times 3 - 5 \\ &= 1 \end{aligned}$$

If $P_k < 0$

$$X_{k+1} = X_k - 1$$

$$Y_{k+1} = Y_k$$

$$P_k = P_k + 2\Delta y$$

If $P_k \geq 0$

$$X_{k+1} = X_k + 1$$

$$Y_{k+1} = Y_k + 1$$

$$P_k = P_k + 2\Delta y - 2\Delta x$$

(10, 7)

k	P_k	X_{k+1}	Y_{k+1}	(X_{k+1}, Y_{k+1})
0	1	9	8	(9,8)
1	$= 1 + 2 \times 3 - 2 \times 5 = -3$	8	8	(8,8)
2	$= -3 + 2 \times 3 = 3$	7	9	(7,9)
3	$= 3 + 2 \times 3 - 2 \times 5 = -1$	6	9	(6,9)
4	$= -1 + 2 \times 3 = 5$	5	10	(5,10)

6. Draw a line for (6,2) and (3,10) using BLA

Solution:

Here, $(X_1, Y_1) = (6, 2)$ & $(X_2, Y_2) = (3, 10)$

$$M = \frac{(10-2)}{(3-6)} = \frac{8}{-3} \text{ -ve slope and } |M| > 1$$

$$\Delta x = |3-6| = 3$$

$$\Delta y = |10-2| = 8$$

Here, Initial $(P_0) = 2\Delta x - \Delta y$

$$= 2 \times 3 - 8$$

$$= -2$$

If $P_k < 0$

$$X_{k+1} = X_k$$

$$Y_{k+1} = Y_k + 1$$

$$P_k = P_k + 2\Delta x$$

If $P_k \geq 0$

$$X_{k+1} = X_k - 1$$

$$Y_{k+1} = Y_k + 1$$

$$P_k = P_k + 2\Delta x - 2\Delta y$$

(6,2)

k	P_k	X_{k+1}	Y_{k+1}	(X_{k+1}, Y_{k+1})
0	-2	6	3	(6,3)
1	$= -2 + 2 \times 3 = 4$	5	4	(5,4)
2	$= 4 + 6 - 16 = -6$	5	5	(5,5)
3	$= -6 + 6 = 0$	4	7	(4,7)
4	$= 0 + 6 - 16 = -10$	4	8	(4,8)
5	$= -10 + 6 = -4$	4	9	(4,9)
6	$= -4 + 6 = 2$	3	10	(3,10)

7. Draw a line for (15,15) and (10,18) using BLA

Solution:

Here, $(X_1, Y_1) = (15, 15)$ & $(X_2, Y_2) = (10, 18)$

$$M = \frac{(18-15)}{(10-15)} = \frac{3}{-5} \text{ -ve slope and } |M| < 1$$

$$\Delta x = |10-15| = 5$$

$$\Delta y = |18-15| = 3$$

Here,

$$\text{Initial } (P_0) = 2\Delta y - \Delta x$$

$$= 2 \times 3 - 5$$

$$= 1$$

If $P_k < 0$

$$X_{k+1} = X_k - 1$$

$$Y_{k+1} = Y_k$$

$$P_k = P_k + 2\Delta y$$

If $P_k \geq 0$

$$X_{k+1} = X_k + 1$$

$$Y_{k+1} = Y_k + 1$$

$$P_k = P_k + 2\Delta y - 2\Delta x$$

(15, 15)

k	P_k	X_{k+1}	Y_{k+1}	(X_{k+1}, Y_{k+1})
0	1	14	16	(14, 16)
1	$= 1 + 2 \times 3 - 2 \times 5 = -3$	13	16	(13, 16)
2	$= -3 + 2 \times 3 = 3$	12	17	(12, 17)
3	$= 3 + 2 \times 3 - 2 \times 5 = -1$	11	17	(11, 17)
4	$= -1 + 2 \times 3 = 5$	10	18	(10, 18)

8. Draw a line for (20,5) and (10,10) using BLA

Solution:

Here, $(X_1, Y_1) = (20, 5)$ & $(X_2, Y_2) = (10, 10)$

$$M = \frac{(10 - 5)}{(10 - 20)} = \frac{5}{-10} \text{ -ve slope and } |M| < 1$$

$$\Delta x = |10 - 20| = 10$$

$$\Delta y = |10 - 5| = 5$$

Here,

$$\begin{aligned} \text{Initial } (P_0) &= 2\Delta y - \Delta x \\ &= 2 \times 5 - 10 \\ &= 0 \end{aligned}$$

If $P_k < 0$

$$X_{k+1} = X_k - 1$$

$$Y_{k+1} = Y_k$$

$$P_k = P_k + 2\Delta y$$

If $P_k \geq 0$

$$X_{k+1} = X_k - 1$$

$$Y_{k+1} = Y_k + 1$$

$$P_k = P_k + 2\Delta y - 2\Delta x$$

(20, 5)

k	P_k	X_{k+1}	Y_{k+1}	(X_{k+1}, Y_{k+1})
0	0	19	6	(19, 6)
1	$= 0 + 2 \times 5 - 2 \times 10 = -10$	18	6	(18, 6)
2	$= -10 + 2 \times 5 = 0$	17	7	(17, 7)

k	P_k	X_{k+1}	Y_{k+1}	(X_{k+1}, Y_{k+1})
3	$= 0 + 2 \times 5 - 2 \times 10 = -10$	16	7	(16, 7)
4	$= -10 + 2 \times 5 = 0$	15	8	(15, 8)
5	$= 0 + 2 \times 5 - 2 \times 10 = -10$	14	8	(14, 8)
6	$= -10 + 2 \times 5 = 0$	13	9	(13, 9)
7	$= 0 + 2 \times 5 - 2 \times 10 = -10$	12	9	(12, 9)
8	$= -10 + 2 \times 5 = 0$	11	10	(11, 10)
9	$= 0 + 2 \times 5 - 2 \times 10 = -10$	10	10	(10, 10)

9. Draw a line for (6,3) and (5,6) using BLA

Solution:

Here, $(X_1, Y_1) = (6, 3)$ & $(X_2, Y_2) = (5, 6)$

$$M = \frac{(6 - 3)}{(5 - 6)} = \frac{3}{-1} \text{ -ve slope and } |M| > 1$$

$$\Delta x = |5 - 6| = 1$$

$$\Delta y = |6 - 3| = 3$$

Here,

$$\begin{aligned} \text{Initial } (P_0) &= 2\Delta x - \Delta y \\ &= 2 \times 1 - 3 \\ &= -1 \end{aligned}$$

If $P_k < 0$

$$X_{k+1} = X_k$$

$$Y_{k+1} = Y_k + 1$$

$$P_k = P_k + 2\Delta x$$

If $P_k \geq 0$

$$X_{k+1} = X_k - 1$$

$$Y_{k+1} = Y_k + 1$$

$$P_k = P_k + 2\Delta x - 2\Delta y$$

(6, 3)

k	P_k	X_{k+1}	Y_{k+1}	(X_{k+1}, Y_{k+1})
0	-1	6	4	(6, 4)
1	$= -1 + 2 \times 1 = 1$	5	5	(5, 5)
2	$= 1 + 2 \times 1 - 2 \times 3 = -3$	5	6	(5, 6)

10. Draw a line for (10,10) and (14,13) using BLA line drawing algorithm.

Solution:

$$\text{Slope: } m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{13 - 10}{14 - 10} = \frac{3}{4} < 1$$

Tips: $m < 1$ conditions ($\Delta x > \Delta y$)

$$d_1 - d_2 > 0$$

$$y_{k+1} = y_k + 1$$

$$x_{k+1} = x_k + 1$$

$$p_{k+1} = p_k + 2\Delta y - 2\Delta x$$

$$d_1 - d_2 < 0$$

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k$$

$$p_{k+1} = p_k + 2\Delta y$$

Initial decision parameter

$$p_0 = 2\Delta y - \Delta x$$

$$\Delta x = 14 - 10 = 4 \quad 2\Delta x = 8$$

$$\Delta y = 13 - 10 = 3 \quad 2\Delta y = 6$$

Initial decision parameter

$$p_0 = 2\Delta y - \Delta x$$

$$= 6 - 4 = 2$$

n	P_k	X_{k+1}	Y_{k+1}
-	-	10	10
0	2	11	11
1	0	12	12
2	-2	13	12
3	4	14	13

11. Use BLA to draw line which end points are A(-3,0), B(4,4)

Solution:

$$x_1 = -3 \quad x_2 = 4$$

$$y_1 = 0 \quad y_2 = 4$$

$$\Delta x = 4 - (-3) = 7 \quad 2\Delta x = 14$$

$$\Delta y = 4 - 0 = 4 \quad 2\Delta y = 8$$

$$m = \frac{4}{7} < 1 \quad [\Delta x > \Delta y]$$

$$p_0 = 2\Delta y - \Delta x = 8 - 7 = 1$$

n	P_k	X_{k+1}	Y_{k+1}
-	-	-3	0
0	1	-2	1
1	-5	-1	1
2	3	0	2

n	P_k	X_{k+1}	Y_{k+1}
3	-3	1	2
4	5	2	3
5	-1	3	3
6	7	4	4

Circle Generation Algorithm

A circle is a geometric shape that is defined as a set of points in a plane that are equidistant from a fixed center point. In simpler terms, it is a perfectly round shape with no corners or edges. The distance from the center of a circle to any point on its boundary is called the radius, and the longest distance across the circle passing through the center is called the diameter. Circle has 8 symmetry (i.e. 8 octants each of 45°)

$$(x-h)^2 + (y-k)^2 = r^2$$

Problems in drawing Circle

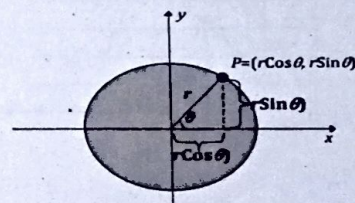
- **Uneven curvature:** One of the most common issues is creating a circle with uneven or wobbly curves. This usually happens due to shaky hands or inconsistent pressure while drawing.
- **Off-center circle:** It can be difficult to accurately position the center point of the circle, resulting in a circle that is not centered properly on the page or object.
- **Jagged edges:** When drawing with certain tools, such as pencils or markers with a blunt tip, the edges of the circle may appear jagged instead of smooth.

Circle equations

- Polar form

$$x = r \cos \theta$$

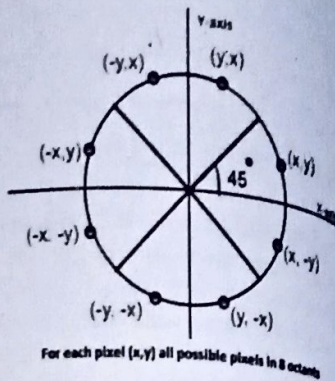
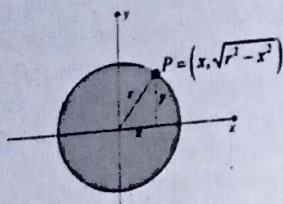
$$y = r \sin \theta \quad (r = \text{radius or circle})$$



Cartesian form

- Use Pythagoras theorem

$$x^2 + y^2 = r^2$$



Bresenham's circle algorithm

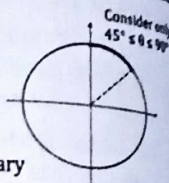
General principle

- The circle function:

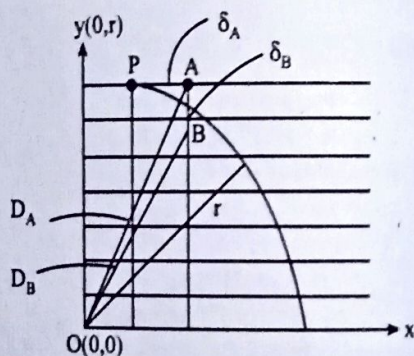
$$f_{\text{circle}}(x, y) = x^2 + y^2 - r^2$$

and

$$f_{\text{circle}}(x, y) = \begin{cases} < 0 & \text{if } (x, y) \text{ is inside the circle boundary} \\ = 0 & \text{if } (x, y) \text{ is on the circle boundary} \\ > 0 & \text{if } (x, y) \text{ is outside the circle boundary} \end{cases}$$



BLA circle drawing algorithm



Here,

$$P(x_i, y_i)$$

$$A(x_i + 1, y_i)$$

$$B(x_i + 1, y_i - 1)$$

Here, x will surely

Increase but y or not decrease.

Let D_A be the distance between origin and A and D_B be the distance between origin and B.

δA and δB are the distance between A & circle surface and B & circle surface.

$$\text{So, } D_A = \sqrt{(x_i + 1)^2 + y_i^2}, D_B = \sqrt{(x_i + 1)^2 + (y_i - 1)^2}$$

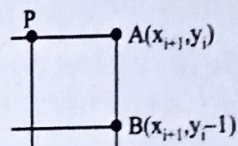
$$\delta A = D_A - r(+ve) \quad \delta B = D_B - r(-ve)$$

To avoid square root in derivation of decision variable we use.

$$\delta A = D_A^2 - r^2, \delta B = D_B^2 - r^2$$

$$\therefore \delta A = (x_i + 1)^2 + y_i^2 - r^2, \delta B = (x_i + 1)^2 + (y_i - 1)^2 - r^2$$

δA is always +ve and δB is always -ve. So we define decision variable d_i as $d_i = \delta A + \delta B$



d_i

$$d_i \geq 0$$

$$\delta_A \geq \delta_B$$

A is for from circle

So choose B(B is closed)

Case 1 $d_i \geq 0$

$$x_{i+1} = x_i + 1$$

$$y_{i+1} = y_i - 1$$

$$d_i < 0$$

$$\delta_A < \delta_B$$

B is for

So choose A(A is closed)

Case 2 $d_i < 0$

$$x_{i+1} = x_i + 1$$

$$y_{i+1} = y_i$$

Starting point is always (0, r)

So, $x = 0, y = r$ then

$$d_i = \delta A + \delta B$$

$$= (x_i + 1)^2 + y_i^2 - r^2 + (x_i + 1)^2 + (y_i - 1)^2 - r^2$$

$$= 1^2 + r^2 - r^2 + 1^2 - r^2 - 2r + 1 - r^2$$

$$= 3 - 2r \text{ (which is initial decision parameter)}$$

Again,

$$d_i = (x_i + 1)^2 + y_i^2 - r^2 + (x_i + 1)^2 + (y_i - 1)^2 - r^2$$

$$= 2(x_i + 1)^2 + 2y_i^2 - 2r^2 - 2y_i + 1$$

At $i+1$ pixel

$$d_{i+1} = 2(x_{i+1} + 1)^2 + 2y_{i+1}^2 - 2r^2 - 2y_{i+1} + 1$$

We know that x will surely increase so

$$x_{i+1} = x_i + 1$$

$$\therefore d_{i+1} = 2(x_i + 1 + 1)^2 + 2y_{i+1}^2 - 2r^2 - 2y_{i+1} + 1$$

$$= 2(x_i + 2)^2 + 2y_{i+1}^2 - 2r^2 - 2y_{i+1} + 1$$

Now,

$$d_{i+1} - d_i = 2(x_i + 2)^2 - 2y_{i+1}^2 - 2r^2 - 2y_{i+1} + 1 - 2(x_i + 1)^2 - 2y_i^2 + 2r^2 + 2y_i - 1$$

On solving:

$$d_{i+1} = d_i + 4x_i + 6 + 2y_{i+1}^2 - 2y_i^2 - 2y_{i+1} + 2y_i$$

Case 1: $d_i > 0$

$$x_{i+1} = x_i + 1$$

$$y_{i+1} = y_i - 1$$

$$d_{i+1} = d_i + 4x_i + 6 + 2(y_i - 1)^2 - 2(y_i - 1) - 2y_i^2 + 2y_i$$

$$= d_i + 4x_i + 6 + 2(y_i^2 - 2y_i + 1) - 2y_i + 2 - 2y_i^2 + 2y_i$$

On solving

$$= d_i + 4(x_i - y_i) + 10$$

Case 2: $d_i < 0$

$$x_{i+1} = x_i + 1$$

$$y_{i+1} = y_i$$

$$d_{i+1} = d_i + y_{i+1} + 6$$

Initial decision parameter

$$x_0 = 0$$

$$y_0 = r$$

$$p_0 = [1^2 + r^2 - r^2] + [1^2 + (r-1)^2 + r^2] = 3 - 2r$$

If $p_i < 0$ then

$$y_{i+1} = y_i$$

$$p_{i+1} = p_i + 4x_i + 6$$

else if $p_i \geq 0$ then

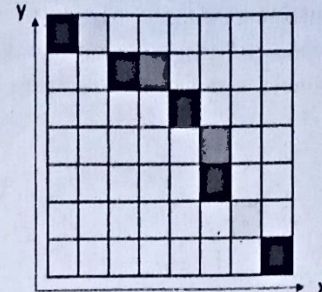
$$y_{i+1} = y_i - 1$$

$$p_{i+1} = p_i + 4(x_i - y_i) + 10$$

$$x_{i+1} = x_i + 1$$

Stop when $x_i \geq y_i$ and determine symmetry points in the other octants

Draw the circle of radius 7 pixel with center (0,0) using Bresenham's circle drawing algorithm.



- We start from a point $(0,r) = (0,7)$
- We know initial decision parameter $= 3 - 2r = 3 - 2 \times 7 = -11$
- We know $d_i \geq 0$ then we use $d_{i+1} = d_i + 4(x_i - y_i) + 10$
- And if $d_i < 0$ then we use $d_{i+1} = d_i + 4x_i + 6$

n	X_i	Y_i	X_{i+1}	Y_{i+1}	Decision parameter	Plot
0	0	7			-11	(0,7)
1	0	7	1	7	-5	(1,7)
2	1	7	2	7	5	(2,7)
3	2	7	3	6	-5	(3,6)
4	3	6	4	6	13	(4,6)
5	4	6	5	5	End	(5,5)

Other points of 1st quadrant:

(6,4), (6,3), (7,2), (7,1), (7,0)

We can find other points as well using 8 way symmetry.

Classwork

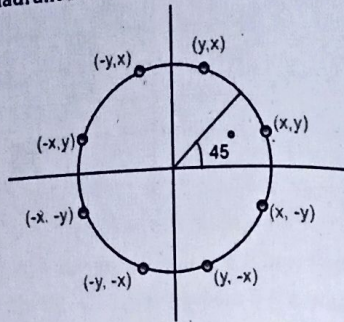
Draw the circle of radius 7 pixel with center (5,6) using Bresenham's circle drawing algorithm.

- Hint start same as (0,0) and then add (5,6)

n	X_i	Y_i	X_{i+1}	Y_{i+1}	Decision parameter	Plot
0	0	7			-11	$(0,7) + (5,6) = (5,13)$
1	0	7	1	7	-5	$(1,7) + (5,6) = (6,13)$
2	1	7	2	7	5	$(2,7) + (5,6) = (7,13)$
3	2	7	3	6	-5	$(3,6) + (5,6) = (8,12)$
4	3	6	4	6	13	$(4,6) + (5,6) = (9,12)$
5	4	6	5	5	End	$(5,5) + (5,6) = (10,11)$

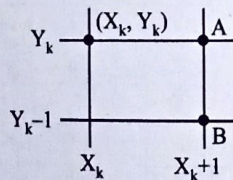
Midpoint circle drawing algorithm

- A circle is a symmetrical figure since the shape of the circle is similar in each quadrant as shown in the figure.

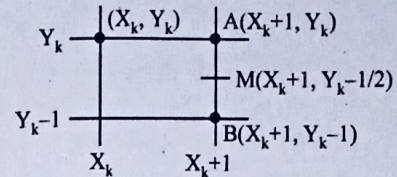


- The calculation of a point (X, Y) in one of octants yields the circle points shown for the other seven octants just by reflection.
- For example if octant (0 to 45) is generated, the second octant can be obtained by reflection through the line $Y=X$ to yield the first quadrant.
- The result in first quadrant are reflected through the line $X=0$, to obtain those in the second quadrant.
- The combined results on the upper semicircle are reflected through the line $Y=0$ to complete the circle.
- The equation of the circle centered at $(0,0)$ is $F_{\text{circle}}(X, Y) = X^2 + Y^2 - r^2$(1)
- The relative position of any point (X,Y) can be determined by checking the sign of the circle function.

$$F_{\text{circle}}(X,Y) \begin{cases} < 0 \text{ if } (X,Y) \text{ is inside the circle boundary} \\ = 0 \text{ if } (X,Y) \text{ is on the circle boundary} \\ > 0 \text{ is outside the circle boundary} \end{cases}$$
- Here, the circle function is the decision parameter and we can set up incremental calculations for the function as we did in the algorithm.
- Let us assume that the pixel location is initially at position of (X_k, Y_k) as shown in figure.



- Now, we determine whether the next plotted pixel location lies at $A(X_k + 1, Y_k)$ or $B(X_k + 1, Y_k - 1)$.
- We take midpoint M between these two candid pixels at sampling position $X_k + 1$ as shown in figure below:

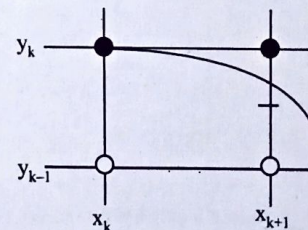


- The decision parameter is given by $P_k = f_{\text{circle}}(X_k + 1, Y_k - 1/2)$
 $= (X_k + 1)^2 + (Y_k - 1/2)^2 - r^2$ equation (2)
- At $(k + 1)^{\text{th}}$ position,
 $P_{k+1} = (X_{k+1} + 1)^2 + (Y_{k+1} - 1/2)^2 - r^2$ equation (3)
- Subtracting equation (2) from (3)
 $P_{k+1} = P_k + (X_{k+1} + 1)^2 - (X_k + 1)^2 + (Y_{k+1} - 1/2)^2 - (Y_k - 1/2)^2$ (4)

Case 1: $P_k < 0$

- The point M lies inside the circle i.e. next pixel position will be at A .
- Then $X_{k+1} = X_k + 1$ and $Y_{k+1} = Y_k$
- Equation (4) becomes as;

$$\begin{aligned} P_{k+1} &= P_k + (X_{k+1} + 1)^2 - (X_k + 1)^2 + (Y_{k+1} - 1/2)^2 - (Y_k - 1/2)^2 \\ &= P_k + (X_k + 1 + 1)^2 - (X_k + 1)^2 + (Y_k - 1/2)^2 - (Y_k - 1/2)^2 \\ &= P_k + (X_k + 2)^2 - (X_k + 1)^2 \\ &= P_k + X_k^2 + 4X_k + 4 - X_k^2 - 2X_k - 1 \\ &= P_k + 2X_k + 3 \\ &= P_k + 2(X_{k+1} - 1) + 3 \\ &= P_k + 2X_{k+1} - 2 + 3 \\ \therefore P_{k+1} &= P_k + 2X_{k+1} + 1 \end{aligned}$$

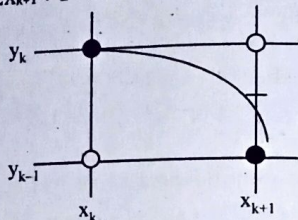


Case 2: $P_k \geq 0$

- The point M lies outside the circle i.e. next pixel position will be at B.
- Then $X_{k+1} = X_k + 1$ and $Y_{k+1} = Y_k - 1$
- Equation becomes as;

$$\begin{aligned}
 P_{k+1} &= P_k + (X_{k+1} + 1)^2 - (X_k + 1)^2 + (Y_{k+1} - 1/2)^2 - (Y_k - 1/2)^2 \\
 &= P_k + (X_k + 1 + 1)^2 - (X_k + 1)^2 + (Y_k - 1 - 1/2)^2 - (Y_k - 1/2)^2 \\
 &= P_k + (X_k + 2)^2 - (X_k + 1)^2 + (Y_k - 3/2)^2 - (Y_k - 1/2)^2 \\
 &= P_k + X_k^2 + 4X_k + 4 - X_k^2 - 2X_k - 1 + Y_k^2 + 9/4 - 3Y_k - Y_k^2 + Y_k - 1/4 \\
 &= P_k + 2X_k + 3 + 2 - 2Y_k \\
 &= P_k + 2(X_{k+1} - 1) + 3 + 2 - 2(Y_{k+1} + 1) \\
 &= P_k + 2X_{k+1} - 2 + 3 + 2 - 2Y_{k+1} - 2 \\
 &= P_k + 2X_{k+1} + 1 - 2Y_{k+1}
 \end{aligned}$$

$$\therefore P_{k+1} = P_k + 2X_{k+1} + 1 - 2Y_{k+1}$$



Initial decision parameter

- The initial decision parameter is obtained by evaluating the circle function at start position $(X_0, Y_0) = (0, r)$.

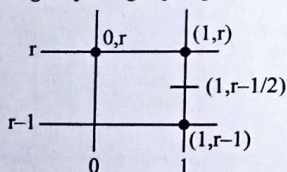
$$\text{i.e. } P_0 = f_{\text{circle}}(1, r - 1/2)$$

$$= 1 + r^2 + 1/4 - r - r^2$$

$$= \frac{5}{4} - r$$

$$= 1.25 - r$$

$$\therefore P_0 = 1 - r \text{ [taking only integer part]}$$



- Draw the circle of radius 9 pixel with center (0,0) using midpoint circle drawing algorithm.

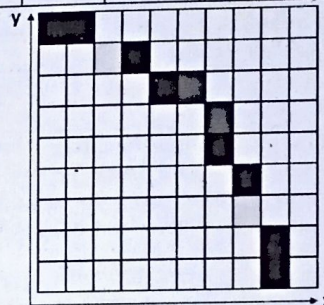
We start from a point $(0, r) = (0, 7)$

We know initial decision parameter $= 1 - r = 1 - 9 = -8$

We know $d_i \geq 0$ then we use $d_{i+1} = d_i + 2(x_i - y_i) + 5$ or $d_{i+1} = d_i + 2(x_{i+1} - y_{i+1}) + 1$

And if $d_i < 0$ then we use $d_{i+1} = d_i + 2x_i + 3$ or $d_i + 2x_{i+1} + 1$

n	X_i	Y_i	X_{i+1}	Y_{i+1}	Decision parameter	Plot
0	0	9			-8	(0,9)
1	0	9	1	9	-5	(1,9)
2	1	9	2	9	0	(2,9)
3	2	9	3	8	-9	(3,8)
4	3	8	4	8	0	(4,8)
5	4	8	5	7	-3	(5,7)
6	5	7	6	7	10	(6,7)
7	6	7	7	6	End	(7,6)



Other points of 1st Quadrant

(7,5), (8,4), (8,3), (9,2), (9,1), (9,0)

We can find other points as well using 8 way symmetry.

Solved Numerical Problems

1. Digitized the circle with radius 10

Solution:

Initial point $= (X_0, Y_0) = (0, r) = (0, 10)$ and origin $= (0, 0)$

Initial decision parameter $(p_0) = 1 - r = 1 - 10 = -9$

From midpoint circle algorithm we have

If $P < 0$

Plot $(x_k + 1, y_k)$

$$P_{k+1} = P_k + 2x_{k+1} + 1$$

Else ($P \geq 0$)

Plot $(x_k + 1, y_k - 1)$

$$P_{k+1} = P_k + 2x_{k+1} - 2y_{k+1} + 1$$

Thus,

Now, the successive decision parameter values and position along the circle path are determined as follow.

k	P_k	x_{k+1}	y_{k+1}	(x_{k+1}, y_{k+1})
0	-9	1	10	(1,10)
1	$-9 + 2 \times 1 + 1 = 6$	2	10	(2,10)
2	$-6 + 2 \times 2 + 1 = -1$	3	10	(3,10)
3	$-1 + 2 \times 3 + 1 = 6$	4	9	(4,9)
4	$6 + 2 \times 4 - 2 \times 9 + 1 = -3$	5	9	(5,9)
5	$-3 + 2 \times 5 + 1 = 8$	6	8	(6,8)
6	$8 + 2 \times 6 - 2 \times 8 + 1 = 6$	7	7	(7,7)
7	$5 + 2 \times 7 - 2 \times 7 + 1 = 5$	8	6	[stop and discard this point since $x > y$]

Hence, the digitized coordinates for circle in first octant are $(x_0, y_0) = (0,10), (x_1, y_1) = (1,10), \dots, (x_7, y_7) = (7,7)$

Note: For each calculated point we need to apply symmetry principle to obtain eight different coordinates one for each of the eight different octants.

2. Digitized a circle with radius $r = 12$ and centered at $(250, 300)$

Solution:

Note: When centre is not origin, we first calculate the octants points of the circle in the same way as the centre at origin then add the given circle center on each calculated pixels.

Here, Initial point $(x_0, y_0) (0, r) = (0, 12)$ and origin $= (0, 0)$

Initial decision parameter $(p_0) = 1 - r = 1 - 12 = -11$

From midpoint circle algorithm we have

If $P < 0$

Plot $(x_k + 1, y_k)$

$$P_{k+1} = P_k + 2x_{k+1} + 1$$

Else ($P \geq 0$)

Plot $(x_k + 1, y_k - 1)$

$$P_{k+1} = P_k + 2x_{k+1} - 2y_{k+1} + 1$$

Thus,

Now, the successive decision parameter values and position along the circle path are determined as follow.

k	P_k	x_{k+1}	y_{k+1}	(x_{k+1}, y_{k+1}) at (0,0)	(x_{k+1}, y_{k+1}) at (250,300)
0	-11	1	12	(1,12)	$(250+1, 300+12)$ $= (251, 312)$
1	$-11 + 2 \times 1 + 1 = -8$	2	12	(2,12)	$(250+2, 300+12)$ $= (252, 312)$
2	$-8 + 2 \times 1 + 1 = -3$	3	12	(3,12)	$(250+3, 300+12)$ $= (253, 312)$
3	$-3 + 2 \times 3 + 1 = 4$	4	11	(4,11)	$(250+4, 300+11)$ $= (254, 311)$
4	$4 + 2 \times 3 - 2 \times 11 + 1 = -9$	5	11	(5,11)	$(250+5, 300+11)$ $= (255, 311)$
5	$-9 + 2 \times 5 + 1 = 2$	6	10	(6,10)	$(250+6, 300+10)$ $= (256, 310)$
6	$2 + 2 \times 6 - 2 \times 10 + 1 = -5$	7	10	(7,10)	$(250+7, 300+10)$ $= (257, 310)$
7	$-5 + 2 \times 7 + 1 = 10$	8	9	(8,9)	$(250+8, 300+10)$ $= (258, 309)$
8	$10 + 16 - 18 + 1 = 9$	9	8	(Discard and stop since $X > Y$)	

Therefore the digitized coordinates are $(x_0, y_0) = (250+0, 300+12) = (250, 312), (x_1, y_1) = (251, 312), \dots, (x_8, y_8) = (258, 312)$

Digitize a circle with a radius 9 and center at (6,7)

Here, the initial decision parameter (P_0)

$$P_0 = 1 - r = 1 - 9 = -8$$

Since, for the midpoint circle algorithm of starting point $(0, r)$ & centre at origin $(0, 0)$ rotating at clockwise direction, we have

If $P < 0$

Plot $(x_k + 1, y_k)$

$$P_{k+1} = P_k + 2x_{k+1} + 1$$

Else ($P \geq 0$)

Plot ($x_k + 1, y_k - 1$)

$$P_{k+1} = P_k + 2x_{k+1} - 2y_{k+1} + 1$$

k	P_k	x_{k+1}	y_{k+1}	(x_{k+1}, y_{k+1}) at (0,0)	(x_{k+1}, y_{k+1}) at (6,7)
0	-8	$0+1=1$	9	(1,9)	$(1+6, 9+7) = (7,16)$
1	$= -8 + 2 \times 1 + 1 = -5$	$1+1=2$	9	(2,9)	$(2+6, 9+7) = (8,16)$
2	$= -5 + 2 \times 2 + 1 = 0$	$2+1=3$	8	(3,8)	$(3+6, 8+7) = (9,15)$
3	$= 0 + 2 \times 3 - 2 \times 8 + 1 = -9$	$3+1=4$	8	(4,8)	$(4+6, 8+7) = (10,15)$
4	$= -9 + 2 \times 4 + 1 = 0$	$4+1=5$	7	(5,7)	$(5+6, 7+7) = (11,14)$
5	$= 0 + 2 \times 5 - 2 \times 7 + 1 = -3$	$5+1=6$	7	(6,7)	$(6+6, 7+7) = (12,14)$
6	$= -3 + 2 \times 6 + 1 = 10$	$6+1=7$	6	(7,6)	$(7+6, 6+7) = (13,13)$

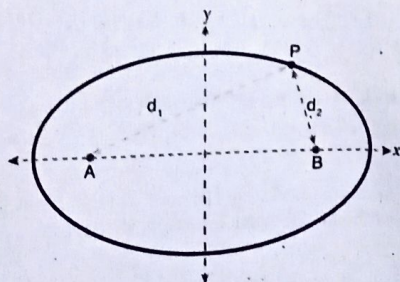
Ellipse Generation Algorithm

An ellipse is the locus of all those points in a plane such that the sum of their distances from two fixed points in the plane, is constant. The fixed points are known as the foci (singular focus), which are surrounded by the curve. The fixed line is directrix and the constant ratio is eccentricity of ellipse. Eccentricity is a factor of the ellipse, which demonstrates the elongation of it and is denoted by 'e' elongation of it and is denoted by 'e'.

Major and Minor Axis

Ellipse is defined by its two-axis along x and y-axis:

- Major axis
- Minor Axis



Here,

P is any point on ellipse,

A & B are the foci,

$PA = d_1$, $PB = d_2$, & $d_1 + d_2 = \text{constant}$

The major axis is the longest diameter of the ellipse (usually denoted by 'a'), going through the center from one end to the other, at the broad part of the ellipse. Whereas the minor axis is the shortest diameter of ellipse (denoted by 'b'), crossing through the center at the narrowest part. Half of major axis is called the semi-major axis and half of minor axis is called semi-minor axis.

Midpoint Ellipse Algorithm

- Ellipse is an elongated circle. Therefore, elliptical curves can be generated by modifying circle drawing procedure to consider the different dimensions of ellipse along the major and minor axes.
- The equation of an ellipse whose centre at origin in standard form is given by

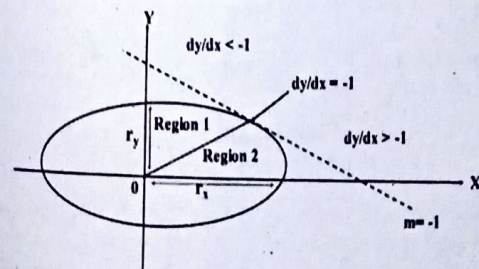
$$\frac{x^2}{r_x^2} + \frac{y^2}{r_y^2} = 1 \text{equation (1)}$$

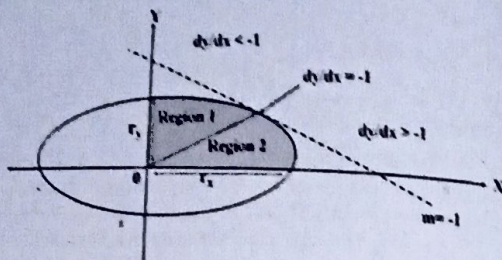
$$\therefore y = \pm r_y \sqrt{1 - \frac{x^2}{r_x^2}}$$

Where r_x and r_y are the radii of the ellipse along the x- and y- direction.

- Midpoint ellipse algorithm is a method for drawing ellipses in computer graphics.
- The method is applied throughout the first quadrant in two parts i.e. region 1 and region 2.
- We are forming these two regions by considering the slope of the curve.

If the slope of the curve is less than -1 then we are in region 1 and when the slope of the curve becomes greater than -1 then we are in region 2 as shown in figure below.

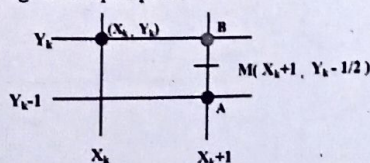




- Starting at $(0, r_y)$, we take unit steps in the x-direction until we reach the boundary of the region 1 and region 2. Then we switch to unit steps in y-direction over the remainder of the curve in the first quadrant.
- The slope of the ellipse is given by

$$\frac{dy}{dx} = -\frac{2ry^2x}{2rx^2y} \dots \text{equation (2) [From equation 1]}$$
- Ellipse function can be defined as:

$$f_{\text{ellipse}}(x, y) = r_y^2x^2 + r_x^2y^2 - r_x^2r_y^2 \dots \text{equation (3)}$$
- From co-ordinate geometry,
 - $f_{\text{ellipse}}(x, y) < 0$ which means (x, y) is inside the ellipse boundary.
 - $f_{\text{ellipse}}(x, y) > 0$ which means (x, y) is outside the ellipse boundary.
 - $f_{\text{ellipse}}(x, y) = 0$ which means (x, y) is on the ellipse boundary.
- At the boundary of region 1 and region 2,
 slope = -1
 or, $\frac{dy}{dx} = -1$
 or, $\frac{-2ry^2x}{2rx^2y} = -1$
 $\therefore 2ry^2x = 2rx^2y$
- Therefore we move out from region 1 whenever
 $2ry^2x \geq 2rx^2y$
- Assuming initial position at (X_k, Y_k) , we determine the next pixel position along the ellipse path.



- Our choices are the pixel positions at $A(X_k+1, Y_k-1)$ and $B(X_k+1, Y_k)$.
- The decision parameter of the midpoint at k^{th} step,

$$P1_k = f_{\text{ellipse}}(X_k+1, Y_k-1/2) = r_y^2(X_k+1)^2 + r_x^2(Y_k-1/2)^2 - r_x^2r_y^2 \dots \text{equation (4)}$$
- At $(k+1)^{\text{th}}$ step, equation (4) becomes as;

$$P1_{k+1} = f_{\text{ellipse}}(X_{k+1}+1, Y_{k+1}-1/2) = r_y^2(X_{k+1}+1)^2 + r_x^2(Y_{k+1}-1/2)^2 - r_x^2r_y^2 \dots \text{equation (5)}$$
- Solving equation (4) and equation (5);

$$P1_{k+1} = P1_k + r_y^2[(X_{k+1}+1)^2 - (X_k+1)^2] + r_x^2[(Y_{k+1}-1/2)^2 - (Y_k-1/2)^2] \dots \text{equation (6)}$$

Case 1: $P1_k < 0$

The midpoint M lies inside the ellipse boundary and the pixel on the scan line Y_k is closer to the ellipse boundary.

So, $X_{k+1} = X_k + 1$

And $Y_{k+1} = Y_k$

The equation (6) becomes as;

$$P1_{k+1} = P1_k + r_y^2[(X_k+1+1)^2 - (X_k+1)^2] + r_x^2[(Y_k-1/2)^2 - (Y_k-1/2)^2]$$

or, $P1_{k+1} = P1_k + 2r_y^2X_k + 3r_y^2$

or, $P1_{k+1} = P1_k + 2r_y^2(X_{k+1}-1) + 3r_y^2$

or, $P1_{k+1} = P1_k + 2r_y^2X_{k+1} - 2r_y^2 + 3r_y^2$

$\therefore P1_{k+1} = P1_k + 2r_y^2X_{k+1} + r_y^2$

Case 2: $P1_k \geq 0$

The midpoint M lies outside the ellipse boundary and the pixel on the scan line Y_k-1 is closer to the ellipse boundary.

So, $X_{k+1} = X_k + 1$

And $Y_{k+1} = Y_k - 1$

Then equation (6) becomes as;

$$P1_{k+1} = P1_k + r_y^2[(X_k+1+1)^2 - (X_k+1)^2] + r_x^2[(Y_k-1-1/2)^2 - (Y_k-1/2)^2]$$

or, $P1_{k+1} = P1_k + 2r_y^2X_{k+1} + r_y^2 + r_x^2[(Y_k-3/2)^2 - (Y_k-1/2)^2]$

or, $P1_{k+1} = P1_k + 2r_y^2X_{k+1} + r_y^2 + r_x^2(2-2Y_k)$

or, $P1_{k+1} = P1_k + 2r_y^2X_{k+1} + r_y^2 + 2r_x^2 - 2r_x^2Y_k$

or, $P1_{k+1} = P1_k + 2r_y^2X_{k+1} + r_y^2 + 2r_x^2 - 2r_x^2(Y_{k+1}+1)$

$\therefore P1_{k+1} = P1_k + 2r_y^2X_{k+1} - 2r_x^2Y_{k+1} + r_y^2$

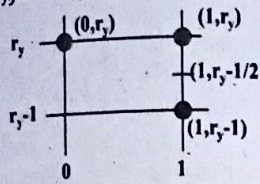
- In region 1, the initial value of decision parameter can be obtained by evaluating the ellipse function at the start position $(X_0, Y_0) = (0, r_y)$.

- Initial decision parameter

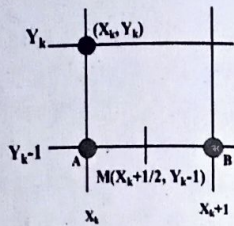
$$\begin{aligned}
 P_{10} &= f_{\text{ellipse}}(1, r_y - 1/2) \\
 &= r_y^2 1^2 + r_x^2 (r_y - 1/2)^2 - r_x^2 r_y^2 \\
 &= r_y^2 + r_x^2 (r_y^2 + 1/4 - r_y) - r_x^2 r_y^2 \\
 &= r_y^2 + r_x^2 r_y^2 + r_x^2/4 - r_y r_x^2 - r_x^2 r_y^2
 \end{aligned}$$

$$\therefore P_{10} = r_y^2 - r_x^2 r_y + r_x^2/4$$

Or simply put $(0, r_y)$ on ellipse equation we will get



- Over region 2, we sample at unit steps along y-direction as shown in figure.



- The decision parameter of the midpoint at k^{th} step,

$$\begin{aligned}
 P_{2k} &= f_{\text{ellipse}}(X_k + 1/2, Y_k - 1) \\
 &= r_y^2 (X_k + 1/2)^2 + r_x^2 (Y_k - 1)^2 - r_x^2 r_y^2 \dots \dots \dots \text{equation (7)}
 \end{aligned}$$

- At $(k + 1)^{\text{th}}$ step, equation (7) becomes as;

$$\begin{aligned}
 P_{2k+1} &= f_{\text{ellipse}}(X_{k+1} + 1/2, Y_{k+1} - 1) \\
 &= r_y^2 (X_{k+1} + 1/2)^2 + r_x^2 (Y_{k+1} - 1)^2 - r_x^2 r_y^2 \dots \dots \dots \text{equation (8)}
 \end{aligned}$$

- Solving equation (7) and (8);

$$\begin{aligned}
 P_{2k+1} &= P_{2k} + r_y^2 [(X_{k+1} + 1/2)^2 - (X_k + 1/2)^2] + r_x^2 [(Y_{k+1} - 1)^2 - (Y_k - 1)^2] \\
 &\dots \dots \dots \text{equation (9)}
 \end{aligned}$$

$$\begin{aligned}
 P_{2k+1} &= P_{2k} + r_y^2 [(X_{k+1} + 1/2)^2 - (X_k + 1/2)^2] + r_x^2 [(Y_{k+1} - 1)^2 - (Y_k - 1)^2] \\
 &= P_{2k} + r_y^2 [(X_{k+1} + 1/2)^2 - (X_k + 1/2)^2] + r_x^2 [(Y_{k+1} - 1)^2 - (Y_k - 1)^2]
 \end{aligned}$$

Case 1: $P_{2k} \leq 0$

The midpoint M lies inside the ellipse boundary and the pixel on the scan line $X_k + 1$ is closer to the ellipse boundary.

$$\text{So, } X_{k+1} = X_k + 1$$

$$\text{And } Y_{k+1} = Y_k - 1$$

The equation (9) becomes as;

$$\begin{aligned}
 P_{2k+1} &= P_{2k} + r_y^2 [(X_k + 1/2)^2 - (X_k + 1/2)^2] + r_x^2 [(Y_k - 1)^2 - (Y_k - 1)^2] \\
 \text{or, } P_{2k+1} &= P_{2k} + r_y^2 [(X_k + 3/2)^2 - (X_k + 1/2)^2] + r_x^2 [(Y_k - 2)^2 - (Y_k - 1)^2] \\
 \therefore P_{2k+1} &= P_{2k} + 2r_y^2 X_{k+1} - 2r_x^2 Y_{k+1} + r_x^2
 \end{aligned}$$

Solution portion skip if u want

$$\begin{aligned}
 P_{2k+1} &= P_{2k} + r_y^2 [(X_k + 1/2)^2 - (X_k + 1/2)^2] + r_x^2 [(Y_k - 1)^2 - (Y_k - 1)^2] \\
 \text{or, } P_{2k+1} &= P_{2k} + r_y^2 [(X_k + 3/2)^2 - (X_k + 1/2)^2] + r_x^2 [(Y_k - 2)^2 - (Y_k - 1)^2] \\
 \text{or, } P_{2k+1} &= P_{2k} + r_y^2 [X_k^2 + 9/4 + 3X_k - X_k^2 - 1/4 - X_k] + r_x^2 [Y_k^2 + 4 - 4Y_k - Y_k^2 - 1 + 2Y_k] \\
 \text{or, } P_{2k+1} &= P_{2k} + r_y^2 (2 + 2X_k) + r_x^2 (3 - 2Y_k) \\
 \text{or, } P_{2k+1} &= P_{2k} + 2r_y^2 + 2r_y^2 X_k + 3r_x^2 - 2r_x^2 (Y_{k+1} + 1) \\
 \text{or, } P_{2k+1} &= P_{2k} + 2r_y^2 + 2r_y^2 X_{k+1} - 2r_y^2 + 3r_x^2 - 2r_x^2 Y_{k+1} - 2r_x^2 \\
 \therefore P_{2k+1} &= P_{2k} + 2r_y^2 X_{k+1} - 2r_x^2 Y_{k+1} + r_x^2
 \end{aligned}$$

Case 2: $P_{2k} > 0$

The midpoint M lies outside the ellipse boundary and the pixel on the scan line X_k is closer to the ellipse boundary.

$$\text{So, } X_{k+1} = X_k$$

$$\text{and, } Y_{k+1} = Y_k - 1$$

Then equation (9) becomes as;

$$\begin{aligned}
 P_{2k+1} &= P_{2k} + r_y^2 [(X_k + 1/2)^2 - (X_k + 1/2)^2] + r_x^2 [(Y_k - 1)^2 - (Y_k - 1)^2] \\
 \text{or, } P_{2k+1} &= P_{2k} + r_x^2 [(Y_k - 2)^2 - (Y_k - 1)^2] \\
 \text{or, } P_{2k+1} &= P_{2k} + r_x^2 [Y_k^2 + 4 - 4Y_k - Y_k^2 - 1 + 2Y_k] \\
 \text{or, } P_{2k+1} &= P_{2k} + r_x^2 (3 - 2Y_k) \\
 \text{or, } P_{2k+1} &= P_{2k} + 3r_x^2 - 2r_x^2 Y_k \\
 \text{or, } P_{2k+1} &= P_{2k} + 3r_x^2 - 2r_x^2 (Y_{k+1} + 1) \\
 \text{or, } P_{2k+1} &= P_{2k} + 3r_x^2 - 2r_x^2 Y_{k+1} - 2r_x^2 \\
 \therefore P_{2k+1} &= P_{2k} - 2r_x^2 Y_{k+1} + r_x^2
 \end{aligned}$$

- In region 2, the initial value of the decision parameter can be obtained by evaluating the ellipse function at the last position in region 1.

$$P_{20} = f_{\text{ellipse}}(X_0 + 1/2, Y_0 - 1)$$

$$\therefore P_{20} = r_y^2 (X_0 + 1/2)^2 + r_x^2 (Y_0 - 1)^2 - r_x^2 r_y^2$$

Where (X_0, Y_0) is the last position in the region 1.

Algorithm

1. Take input radius along x axis and y axis and obtain center of ellipse.
2. Initially, we assume ellipse to be centered at origin and the first point as: $(x_0, y_0) = (0, r_y)$.
3. Obtain the initial decision parameter for region 1 as:
 $P_{10} = r_y^2 - r_x^2 r_y + r_x^2 / 4$
4. For every X_k position in region 1 until $(2r_y^2 X_{k+1} > 2r_x^2 Y_{k+1})$
 - Check if $P_{1k} < 0$ then the next point is (X_{k+1}, Y_k) and $P_{1k+1} = P_{1k} + 2r_y^2 X_{k+1} + r_y^2$
 - Otherwise, the next point is (X_{k+1}, Y_{k-1}) and $P_{1k+1} = P_{1k} + 2r_y^2 X_{k+1} - 2r_x^2 Y_{k+1} + r_y^2$
5. Obtain the initial decision parameter for region 2 using the last point (X_0, Y_0) of region 1 as:
 $P_{20} = r_y^2 (X_0 + 1/2)^2 + r_x^2 (Y_0 - 1)^2 - r_x^2 r_y^2$
6. At each Y_k in region 2 starting at $k = 0$ perform the following tasks.
 - Check if $P_{2k} < 0$ the next point is $(X_k + 1, Y_k - 1)$ and $P_{2k+1} = P_{2k} + 2r_y^2 X_{k+1} - 2r_x^2 Y_{k+1} + r_x^2$
 - Otherwise, the next point is $(X_k, Y_k - 1)$ and $P_{2k+1} = P_{2k} - 2r_x^2 Y_{k+1} + r_x^2$
7. Repeat step 6 until we get one complete quadrant of ellipse i.e. the pixel point $(r_x, 0)$.
8. Now obtain the symmetric points in the three quadrants and plot the coordinate value as: $X = X + X_c, Y = Y + Y_c$

Draw an Ellipse with radii $r_x = 8$ and $r_y = 6$ with center $(0,0)$

Quick hints

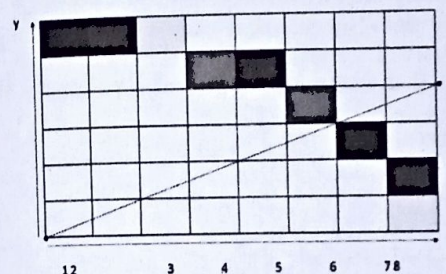
- We have to find all the points of ellipse (i.e. region 1 and region 2)
 - We have to start with $(0, r_y)$ [point of region 1] and ended with $(r_x, 0)$ [point of region 2]
 - Find $dy = 2Y_{k+1} \times r_x^2$ and $dx = 2X_{k+1} \times r_y^2$ and compare every time. If $dy > dx$ then we are in region 1 so use all the formula related to region 1
- Initial decision parameter of region 1: $P_{10} = r_y^2 - r_x^2 r_y + r_x^2 / 4$
- If $P_{1k} < 0$ then the next point is (X_{k+1}, Y_k) and $P_{1k+1} = P_{1k} + 2r_y^2 X_{k+1} + r_y^2$
 - Otherwise, the next point is (X_{k+1}, Y_{k-1}) and $P_{1k+1} = P_{1k} + 2r_y^2 X_{k+1} - 2r_x^2 Y_{k+1} + r_y^2$

When $dx > dy$ we shift from region 1 to region 2 and use the formula related to region 2

Initial decision parameter of region 2: $P_{20} = r_y^2 (X_0 + 1/2)^2 + r_x^2 (Y_0 - 1)^2 - r_x^2 r_y^2$

- Check if $P_{2k} < 0$ the next point is $(X_k + 1, Y_k - 1)$ and $P_{2k+1} = P_{2k} + 2r_y^2 X_{k+1} - 2r_x^2 Y_{k+1} + r_x^2$
- Otherwise, the next point is $(X_k, Y_k - 1)$ and $P_{2k+1} = P_{2k} - 2r_x^2 Y_{k+1} + r_x^2$

n	X_k	Y_k	Decision parameter (p)	X_{k+1}	Y_{k+1}	$dx = 2X_{k+1} \times r_y^2$	$dy = 2Y_{k+1} \times r_x^2$	Plot (x,y)	
0	0	6	$P_{10} = -332$	1	6	72	768	(1,6)	Region 1
1	1	6	$P_{11} = -224$	2	6	144	768	(2,6)	
2	2	6	$P_{12} = -44$	3	6	216	768	(3,6)	
3	3	6	$P_{13} = 208$	4	5	288	640	(4,5)	
4	4	5	$P_{14} = -108$	5	5	360	640	(5,5)	
5	5	5	$P_{15} = 288$	6	4	432	512	(6,4)	
6	6	4	$P_{16} = 244$	7	3	504	384	(7,3)	Region 2
7	7	3	$P_{20} = -23$	8	2	576	256	(8,2)	
8	8	2	$P_{21} = 361$	8	1	576	128	(8,1)	
9	8	1	$P_{22} = 297$	8	0	End	End		



Region 1 points	
(0,6)	Starting point
(1,6)	
(2,6)	
(3,6)	
(4,5)	
(5,5)	
(6,4)	
Region 2 points	
(7,3)	
(8,1)	
(8,0)	ending point

Area Filling Scan-line Algorithm

Filling is the process of "coloring in" a fixed area or region. A standard output primitive in general graphics is a solid color or patterned polygon area.

There are two basic approaches to area filling on a raster system.

- One way to fill an area is to determine the overlap intervals for scanlines across the area, i.e., the scan line fill algorithm.
 - Another method for area filling is to start from a given interior position and point outward from this point until we encounter the specified boundary edges.
- Scanline Fill:** The scanline fill algorithm scans each horizontal line of a polygon and determines the intersections with the polygon edges. It then fills the pixels between the intersections. This technique works well for convex and concave polygons and can handle gradients or patterns.
 - Boundary Fill:** The boundary fill algorithm starts at a given seed point inside a shape and recursively fills the area until it encounters a boundary or a different color. This technique is useful for filling closed shapes with solid colors or patterns.
 - Flood Fill:** The flood fill algorithm is similar to boundary fill, but it fills an entire connected region based on a starting point and a target color. It continues to fill adjacent pixels until it reaches the boundary or encounters a different color.
 - Seed Fill:** Seed fill is a variation of the flood fill algorithm where multiple seed points are specified, and each seed point fills its connected region. This technique is useful for filling complex shapes or regions with different colors.
 - Texture Mapping:** Texture mapping involves mapping a 2D image or texture onto a 3D surface. This technique applies the texture to the surface's polygons, filling the areas with the texture's color or pattern. It is commonly used in 3D rendering and computer games.

Scan line polygon fill algorithm

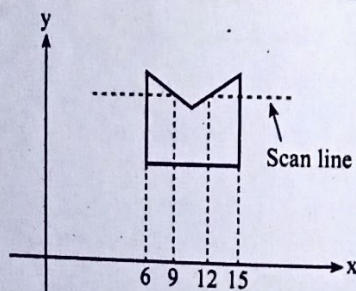


Fig.: (a)

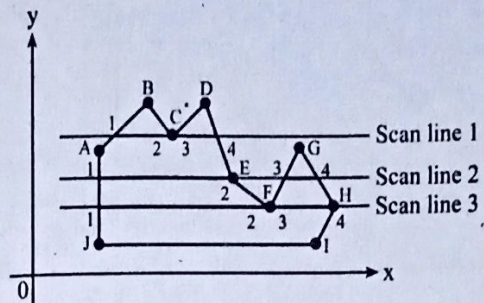


Fig.: (b) Intersection points along the scan line that intersect polygon vertices

Figure (a) illustrates the scan line algorithm for filling of the polygon. For each scan line crossing a polygon, this algorithm locates the intersection points of the scan line with the polygon edges. These intersection points are then sorted from left to right, and the corresponding positions between each intersection pair are set to the specified fill color.

The important task in the scan line algorithm is to find the intersection points of the scan line with the polygon boundary. When intersection points are even, they are sorted from left to right, paired and pixels between paired points are set to the fill color. But in some cases, the intersection point is a vertex. When a scan line intersects a polygon vertex special handling is required to find the exact intersection points. To handle such cases, we must look at the other endpoints of the two line segments of the polygon which meet at this vertex. If these points lie on the same (up or down) side of the scan line, then the point in question counts as an even number of intersections. If they lie on opposite sides of the scan line, then the point is counted as a single intersection. This is illustrated in Figure (b).

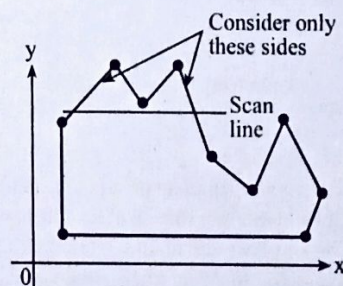


Fig.: (c) Consider only the sides which intersect the scan line

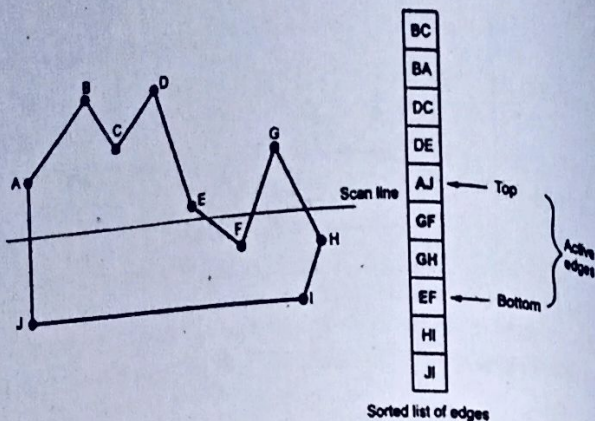
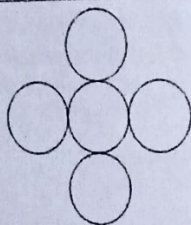


Fig.: (d) Sorted edges of the polygon with active edges

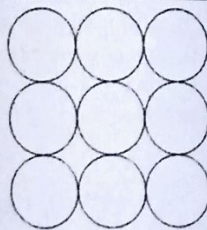
Many times it is not necessary to compute the x intersections for scan line with every polygon side. We need to consider only the polygon sides with endpoints straddling the current scan line. See figure (c).

It will be easier to identify which polygon sides should be tested for x-intersection, if we first sort the sides in order of their maximum y value. Once the sides are sorted we can process the scan lines from the top of the polygon to its bottom producing an active edge list for each scan line crossing the polygon boundaries. The active edge list for a scan line contains all edges crossed by that scan line. The figure (d) shows sorted edges of the polygon with active edges.

Boundary Fill Techniques



Four Connected



Eight Connected

4-Connected:

In a 4-connected system, a pixel is considered to be connected to its four immediate neighbors: the pixel to the north, south, east, and west. These are the four cardinal directions on a grid. The concept of 4-connectedness implies that diagonal neighbors are not considered connected.

For example, in a 2D grid, if we have a pixel at coordinates (x, y) , its 4-connected neighbors would be $(x, y+1)$ to the north, $(x, y-1)$ to the south, $(x+1, y)$ to the east, and $(x-1, y)$ to the west.

8-Connected:

In an 8-connected system, a pixel is considered to be connected to its eight neighboring pixels, which include both cardinal and diagonal directions. In addition to the north, south, east, and west neighbors, it also includes the four diagonal neighbors.

Continuing from the previous example, in a 2D grid, if we have a pixel at coordinates (x, y) , its 8-connected neighbors would be $(x, y+1)$ to the north, $(x, y-1)$ to the south, $(x+1, y)$ to the east, $(x-1, y)$ to the west, $(x+1, y+1)$ to the northeast, $(x+1, y-1)$ to the southeast, $(x-1, y-1)$ to the southwest, and $(x-1, y+1)$ to the northwest.

Boundary Fill Algorithm

This is a recursive algorithm that begins with a starting pixel called a seed, inside a region and continuous painting toward the boundary. The algorithm checks to see if this pixel is a boundary pixel or has already filled. If answer is No, it fills the pixel and makes a recursive call to itself using each and every neighbouring pixel as a new seed. If the answer is yes, the algorithm simply returns to its caller.

A boundary - fill procedure accepts as input the coordinates of an interior point (x, y) , a fill colour and a boundary colour. Starting from (x, y) , the procedure tests neighbouring positions to determine whether they are of the boundary colour. If not, they are painted with the fill colour and their neighbours are tested. This process continues until all pixels up to the boundary colour for the area have been tested. There may be two methods for preceding to neighbouring pixels from the seed point.

1. 4-connected
2. 8-connected

The following procedure illustrates a recursive method for filling a 4-connected area. The boundary fill method requires the coordinates of a starting point, a fill colour and background colour as arguments.

Void boundary fill (int x, int y, int fill, int boundary)

```
{
    int current = get pixel (x, y);
    if ((current != boundary) & current != fill)
    {
        SetColor (fill);
    }
}
```



```

SetPixel (x, y);
boundary Fill (x + 1, y, fill, boundary);
boundary Fill (x - 1, y, fill, boundary);
boundary Fill (x, y + 1, fill, boundary);
boundary Fill (x, y - 1, fill, boundary);
}
}

```

Recursive boundary-fill algorithms may not fill regions correctly if some interior pixels are already displayed in the fill colour. This occurs because the algorithm checks next pixels both for boundary colour and for fill colour. Encountering a pixel with the fill colour can cause a recursive branch to terminate, leaving other interior pixels unfilled. To avoid this, we can first change the colour of any interior pixels that are initially set to the fill colour before applying the boundary-fill procedure.

Also, since this procedure requires considerable stacking of neighbouring points, more efficient methods are generally employed. These methods fill horizontal pixel spans across scan line, instead of proceeding to 4-connected or 8-connected neighbouring points. Then we need only stack a beginning position for each horizontal pixel span, instead of stacking all unprocessed neighbouring positions around the current position. Starting from the initial interior point with this method, we first fill in the contiguous span of pixels in this starting scan line. Then we locate and stack starting positions for spans on the adjacent scan lines, where spans are defined as the contiguous horizontal string of positions bounded by pixels displayed in the area border colour. At each subsequent step, we unstack the next start position and repeat the process.

The boundary fill algorithm, is having limitation. It fills the polygon having unique boundary colour. If the polygon is having boundaries with different colours then this algorithm fails.

Flood fill Algorithm

This algorithm also begins with a seed (starting pixel) inside the region. It checks to see if the pixel has the regions original color. If the answer is yes, it fills the pixel with a new colour and uses each of the pixel's neighbour as a new seed in a recursive call. If answer is no, it returns to the caller.

This method shares great similarities in its operating principle with the boundary fill algo. It is particularly useful when the region to be filled has no uniformly coloured boundary i.e., fill in an area that is not defined within a single colour boundary. So in this we can point such areas by

replacing a specified interior colour instead of searching for a boundary colour value. If the area we want to paint has more than one interior colour, we can first reassign pixel values, so that all interior points have the same colour. Using either a 4-connected or 8-connected approach, we then step through pixel positions until all interior points have been repainted.

The following procedure flood fills a 4-connected region, recursively, starting from the input position.

```

Void floodfill (int x, int y, int fill, int oldcolor)

```

```

{
    if (getpixel (x, y) = oldcolor)
    {
        SetColor (fill);
        SetPixel (x, y);
        Floodfill (x + 1, y, fill, oldcolor);
        Floodfill (x - 1, y, fill, oldcolor);
        Floodfill (x, y + 1, fill, oldcolor);
        Floodfill (x, y - 1, fill, oldcolor);
    }
}

```

We can modify procedure flood fill to reduce the storage requirements of the stack by filling horizontal pixel spans as boundary-fill algorithm. In this approach, we stack only the beginning positions for those pixel spans having the value oldcolor. The steps in this modified algorithm are similar to those for boundary fill. Starting at the first position of each span, the pixel values are replaced until a value other than oldcolor is encountered.